uc3m | Universidad Carlos III de Madrid

Bachelor in Data Science and Engineering.
2024-2025

Bachelor Thesis

"AI-Driven Digital Assistant for Task Optimization and Automated Support"

Alejandro Taboada Esteban

Company tutor: Ignacio Satrústegui Higuero
University tutor: Fernando Díaz de María
Leganés, June 2025

This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

ABSTRACT

This thesis looks at the design, development, and deployment of an AI-driven digital assistant built for day-to-day work at Missio IA, a young company focused on enterprise automation. The assistant links up with the main company tools, namely Gmail, Google Calendar, Trello, GitHub, and Google Drive through Model Context Protocol servers, so it can collect information without extra glue code.

The assistant has two main jobs. During off-hours it scans all the data and writes a personalised plan for the next working day. When everyone is online, it switches to an interactive mode that feels close to a traditional chatbot yet stays inside the firm's own workflows and files. It also scans every received email, finding potential clients so it can browse for information about them and formulate preliminary insights before any request is made. Thanks to this mix, coordination effort drops, hidden blockers tend to surface earlier, and everyday decisions go faster.

On top of that, the built system, named Missio OS, brings capabilities and automations that no current competitor can offer, thanks to the MCP integrations The assistant solves 89.65% of tasks on the first try, comfortably clearing the 85% "human-reliability" barrier. This extra depth means the project is not just another digital helper; it moves the firm one step closer to the vision of an autonomous organisation and stands out as a clear leap in innovation.

The thesis also studies how large language models can handle real tools instead of just plain text. Tests cover success rate, response time, and running cost, with the target of matching or even passing the reliability one would expect from a human.

Finally, the project stands as a concrete piece of applied innovation in a real business setting and should help both Missio IA and any reader curious about rolling out an AI assistant in a start-up environment.

Keywords: Language Models, Digital Assistant, Task Planning, Workflow Automation, Model Context Protocol, Applied AI

DEDICATION

I would like to thank everyone who has supported me while developing this project, especially my parents, who have been supporting me all my life and reminding me of what I am capable of.

I would like to thank my brother Jacobo too, who is a true support and pillar for me.

Also, I would like to thank the rest of my family, because they really believe in me and make me a happy person.

I would also like to thank, Luis Alarcón de la Lastra and Ignacio Satrústegui, my tutors from Missio IA, who have given me space to test ideas and learn from the outcome, and have trusted me from day one and guided me through all this process.

I would like to thank my tutor from university too, Fernando Díaz de María, because he has assisted me with all the formalities and procedures that I would not have been able to make without his help.

And I would like to thank my friend Sergio Rodríguez, who has shared the experience with me at Missio IA, and a lot of moments and learnings developing projects together.

And finally, I would like to thank the ones that have been with me everyday and made this journey a lot easier by helping me having fun, my friends from Madrid and Coruña.

Thank you very much to all of you

CONTENTS

ACRONYMS xv
1. INTRODUCTION 1
1.1. Context and Motivation 1
1.2. Problem Statement 2
1.3. Objectives 3
1.4. Scope of the Project 4
2. BACKGROUND AND STATE OF THE ART 5
2.1. Foundations of Artificial Intelligence 5
2.1.1. Learning Paradigms 5
2.1.2. Neural Architectures 6
2.1.3. Scaling Laws and Large Language Models 6
2.1.4. Evolution of AI: From Symbolic to Neural Approaches 8
2.2. Transformer-Based Language Models 8
2.2.1. Autonomous Reasoning and Chain-of-Thought 9
2.2.2. Advanced Prompting Techniques 9
2.2.3. Pre-training and Fine-tuning Strategies 10
2.3. Large Language Models and Tool Use 14
2.3.1. From Function Calling to MCP 14
2.3.2. Interoperability Standards 15
2.4. Enterprise Digital Assistants 16
2.4.1. Commercial Solutions 16
2.4.2. Productivity Impact 17
2.4.3. Autonomy and Human-in-the-Loop 19
2.5. Task Planning and Optimization Approaches 20
2.5.1. Prompt-Based Reasoning (CoT and ToT) 20
2.5.2. Hybrid LLM + External Solvers 20
2.5.3. Reinforcement-Learning-Based Scheduling 21
2.5.4. Multi-Agent Collaboration 24

2.6. Synthesis and Research Gaps 25
3. SYSTEM REQUIREMENTS AND DESIGN 29
3.1. Functional Requirements 29
3.1.1. Client Communication and Information Management 29
3.1.2. Internal Workflow Automation and Optimization 30
3.1.3. Intelligent Assistant and User Interaction 32
3.2. Non-Functional Requirements 33
3.2.1. Performance 33
3.2.2. Reliability and Availability 34
3.2.3. Scalability 34
3.2.4. Security 35
3.2.5. Privacy 35
3.2.6. Maintainability and Extensibility 36
3.2.7. Usability 36
3.2.8. Cost 37
3.3. Architecture Overview 37
3.3.1. High-level system overview 37
3.3.2. Core Architectural Components 39
3.3.3. Key System Workflows 41
3.4. Technology Stack 41
3.5. Security and Privacy 43
4. IMPLEMENTATION 45
4.1. Connector Modules (MCPs) 45
4.1.1. Gmail MCP 45
4.1.2. Google Drive MCP 46
4.1.3. Trello MCP 46
4.1.4. GitHub MCP 46
4.1.5. Google Calendar MCP 46
4.1.6. FireCrawl MCP 46
4.2. Agent Orchestration 47
4.2.1. Design Evolution and Iteration 47

4.2.2. Prompting Strategies for Agent Behaviour. 47
4.2.3. Planner/BlockerDetector 48
4.2.4. EmailOpportunityDetector 48
4.2.5. File-Mediated Communication 48
4.3. Backend Services. 48
4.4. Front-End and Authentication. 51
4.5. DevOps and Monitoring 52
4.6. User Feedback and Iterative Improvements 53
5. EVALUATION 55
5.1. Evaluation Methodology 55
5.2. Models Compared 57
5.3. Metrics 57
5.4. Results 58
5.5. Discussion of Findings. 60
6. DISCUSSION 62
6.1. Interpretation of Results 62
6.2. Comparison with Market Alternatives 64
6.3. Limitations 65
6.4. Broader Implications and Future Impact 66
6.5. Technical Learnings and Challenges Addressed. 67
7. CONCLUSIONS AND FUTURE WORK 69
7.1. Main Contributions 69
7.2. Future Work 71
BIBLIOGRAPHY. 73

LIST OF FIGURES

2.1 AI hierarchy 7
2.2 RLFH Pipeline 11
2.3 RL Loop Diagram 22
2.4 RL Algorithms taxonomy 22
2.5 Cloud RL Workflow Diagram 23
2.6 LLM - RL Agent Interaction 24
2.7 Trade-off in Cost-aware RL 24

3.1 High-Level Architecture of Missio OS. 38

4.1 Sequence Diagram of POST/connect Endpoint. 50
4.2 Sequence Diagram of POST/chat Endpoint 50
4.3 Missio OS Interface 52

5.1 GPT-4.1 success rate by task difficulty. 59
5.2 GPT-4o success rate by task difficulty. 59
5.3 o4-mini success rate by task difficulty. 60

LIST OF TABLES

2.1 Comparison of Popular Digital Assistants with Enterprise Extensions. . . 17

2.2 Comparison of Enterprise Digital Assistants and Missio OS 28

5.1 Final Evaluation Results 58

ACRONYMS

API Application Programming Interface. 5, 14, 16, 25, 35, 37, 39, 40, 42, 45, 47, 52, 57, 65–69, 71

BERT Bidirectional Encoder Representations from Transformers. 9

EDA Enterprise Digital Assistant. 16

GDPR General Data Protection Regulation. 35, 43

IDE Integrated Development Environment. 15, 16

MCP Model Context Protocol. iii, vii, viii, 2, 5, 14–16, 25–27, 33, 36–40, 42, 45–48, 51, 53, 55, 56, 60–72

RAG Retrieval-Augmented Generation. 13

RL Reinforcement Learning. 5, 21–24

RLHF Reinforcement Learning from Human Feedback. 11–13

SaaS Software as a Service. 1

SDK Software Development Kit. 52, 57

1. INTRODUCTION

1.1. Context and Motivation

Missio IA is a Madrid-based start-up specialised in applying artificial-intelligence techniques to improve business efficiency. Its activities combine two lines of work. On the one hand, the company offers ready-made Software-as-a-Service (SaaS) products that clients can adopt with minimal integration effort. On the other hand, it develops tailored automation projects that address more complicated fields such as document processing, customer support, or content generation. The goal of all solutions: to save organisations time and money by reducing manual effort and accelerating decision making.

The company is run by a small multidisciplinary team that covers engineering, design, product management, and client relations. In this setting, each member handles several responsibilities, moving in the same week from writing production code to refining product mock-ups and presenting progress to customers. The innovation cycle is shortened by this adaptability, but more coordination work needs to be done. Trello is used to keep track of tasks, GitHub is used to manage source code, Google Drive is used to store project documents, Gmail is used to communicate with clients, and Google Calendar is used to schedule meetings. Shifting attention among these separate systems absorbs a noticeable share of every working day and can distract effort from other activities.

A second motivation, beyond pure efficiency, is the company's culture of experimentation. Missio IA exists to test ideas that five years ago would have sounded like science fiction: giving an AI system the initiative to organise work as if it were another employee. Building such a tool for our own use lets us explore the edge of what is technically possible while keeping risks manageable. This pursuit of practical innovation is at the heart of Missio IA's culture and provides an ideal framework for my final-year engineering project.

Two recurrent problems illustrate the situation. First, blockers may remain hidden when a code review is delayed or when a design specification becomes inconsistent with the latest implementation. Second, daily meetings often extend beyond their intended duration because the team needs to consolidate priorities, arrange ad-hoc client requests, and resolve calendar conflicts. A preliminary time-tracking exercise carried out in January 2025 indicated that reducing such friction would recover around four engineering hours per week, a saving that is significant for a company of this size.

To address these challenges the present project proposes an internal digital assistant that functions as a virtual operations officer. The assistant runs autonomously outside office hours, collects status information from all relevant platforms, and produces an optimised day plan for every employee. It also identifies latent dependencies so that potential delays can be mitigated before they affect delivery dates.

During business hours the same system is available as an interactive assistant, comparable in usability to tools such as ChatGPT. Employees can ask questions in natural language, request ad-hoc summaries, generate drafts, or trigger specific automations without leaving the company dashboard. In this way the assistant becomes a regular work instrument, augmenting everyday decision-making rather than acting only in the background.

Recent advances in tool-using large language models make this approach feasible. The Model Context Protocol (MCP), introduced in late 2024, provides a model-agnostic standard for invoking external services while maintaining conversational context [1], [2]. With MCP, numerous heterogeneous connectors can be orchestrated by a single instance of an AI model without the need for additional integration code. This feature eliminates the conventional $N \times M$ adapter issue and streamlines maintenance, which is crucial for a start-up with limited resources.

In addition to providing immediate efficiency gains, an internally validated assistant would enhance Missio IA's market proposition i. Demonstrating that the company can automate its own operations lends credibility when offering similar technology to clients. In addition, the assistant serves as a controlled test-bed for new connectors, prompting strategies, and cost-optimisation measures that can later be incorporated into commercial products.

From a research perspective the project examines whether a single language-model agent can orchestrate multiple MCP connectors reliably, generate daily plans that approach or exceed human scheduling quality, and detect hidden blockers with adequate recall and precision. Answering these questions contributes empirical data to a field that is evolving rapidly but still lacks documented real-world deployments of comparable scope.

In summary, the compact structure of Missio IA amplifies the effect of coordination overhead while providing the agility needed to deploy cutting-edge AI solutions. The convergence of MCP, capable yet affordable models such as OpenAI's o4-mini, and a homogeneous cloud infrastructure offers a timely opportunity to implement and evaluate an autonomous assistant that improves internal efficiency and illustrates the next generation of enterprise digital-assistant technology.

1.2. Problem Statement

Despite the rapid adoption of artificial intelligence in many business areas, no commercially available assistant offers the level of end-to-end support that Missio IA requires. Current products excel at isolated tasks, such as drafting an e-mail, proposing a meeting slot, or summarising a document, but they rarely operate across the whole information space at once. Code reviews are in one system, customer requests are in another, and design modifications are in a third, creating a fragmented critical context. Workers transmit status updates and waste valuable time acting as human routers.

The impact is amplified in a start-up, where the number of workers is limited and roles are broad. Because the same person alternates between engineering, design, quality

assurance, and customer presentations, interruptions caused by missing context break concentration and delay delivery. Even minor inconsistencies, like an outdated specification or an overlooked calendar event can propagate into rework or missed deadlines. Internal measurements performed in early 2025 showed that coordination overhead consumed roughly one working day every fortnight, a cost not every company can afford.

Three requirements must be met for a solution to be genuinely successful. Initially, it has to have ongoing, authorised access to all operational data sources, including shared documents, calendar events, task trackers, code repositories, and email threads. Secondly, it must use the combined knowledge space to reason in order to find dependencies, predict impending bottlenecks, and provide an optimized strategy for the upcoming work period without being asked. In order to effortlessly integrate into everyday routines rather than adding another dashboard, it must, third, provide an interactive interface that staff members can consult, similar to any contemporary conversational system.

None of the enterprise assistants in use today meet these requirements as of this writing. Commercial platforms tend to be constrained to a small set of features, sometimes restricted to the ecosystem of a single vendor, and rarely take proactive steps. Research prototypes exhibit some of the intended behavior, but they are limited to laboratory benchmarks or proprietary test beds and don't demonstrate much reliability in the actual world. Therefore, the challenge of creating, deploying, and assessing an autonomous assistant that can monitor a business's whole operational state, step in before problems worsen, and assist staff members via a conversational interface is still unresolved. If small, resource-constrained organizations are to experience the same productivity improvements as larger enterprises with specialized operations teams, this gap must be closed.

1.3. Objectives

The objectives of this project are the following:

- Create and deploy a digital assistant that is always ready to use and has authorized access to Google Calendar, GitHub, Gmail, Trello, Google Drive, and the FireCrawl search engine.
- Reach a task-success rate greater than eighty five percent, a threshold chosen to match or exceed the performance that is estimated for a human team member, so that staff can rely on the assistant with confidence.
- Provide an intuitive web interface that every employee can use without prior knowledge of artificial-intelligence systems, offering clear authentication, concise responses and minimal configuration overhead.
- Ensure that operating costs stay within a sustainable ceiling, so each routine action handled by the assistant is less expensive than the equivalent manual effort.

- Enable the assistant to detect and report hidden blockers in real time and to accelerate internal workflows by automatically proposing and executing the necessary follow-up actions.

1.4. Scope of the Project

This project covers the design, implementation, and initial evaluation of an AI-driven assistant tailored to Missio IA's internal workflow. Work is limited to the integration of six data sources, that are Gmail, Trello, Google Drive, GitHub, Google Calendar, and FireCrawl for web research, through the Model Context Protocol. The final product includes a nightly planning agent that runs without supervision, a set of blocker-detection routines, and a web front-end that allows any team member to consult the assistant. All development is carried out using Google Cloud services and tested with the company's own accounts.

The project does not attempt to create a new language model. It relies on commercially available models, focusing instead on orchestration, security, and usability. Ongoing improvements in the model landscape allow the assistant to benefit from better models without having to focus on that aspect. Mobile clients, , large-scale load testing, and integration with financial or HR systems remain future work. Evaluation is confined to task-success rate, latency, cost per request, and qualitative feedback from Missio IA staff, broader market studies are excluded. The outcome should be a dependable assistant that reduces coordination overhead while staying within the operational constraints of a start-up.

2. BACKGROUND AND STATE OF THE ART

Understanding the recent advances that link artificial-intelligence theory with day-to-day software practice is essential before discussing the design choices of this project. The chapter therefore opens with the foundations of modern large language models and the principles that govern their interaction with external tools. It then traces the evolution from early function-calling Application Programming Interfaces (APIs) to the Model Context Protocol (MCP) and reviews the main interoperability standards that seek to make tool integration model-agnostic. Finally, it surveys enterprise digital assistants, task-planning techniques and the open research questions that motivate the system proposed in later chapters.

2.1. Foundations of Artificial Intelligence

Artificial intelligence (AI) can be understood as the study of agents that perceive their environment, process information, and select actions that maximise a stated objective [3]. Modern AI practice is dominated by data-driven approaches in which statistical models learn patterns directly from examples.

2.1.1. Learning Paradigms

Three learning paradigms frame the majority of current systems:

- Supervised learning. A model receives labelled pairs (x, y) and learns a mapping $f : x \to y$ by minimising a loss such as cross-entropy or mean-squared error [4]. Typical applications include image classification and machine translation.
- Unsupervised learning. Here the goal is to uncover latent structure in unlabelled data, often by maximising likelihood or employing contrastive objectives. Clustering, topic modelling, and auto-encoding fall under this heading [5].
- Reinforcement learning (RL). An agent interacts with an environment modelled as a Markov Decision Process (S, A, P, R, γ) and seeks to maximise the expected return

  $$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

  where $\gamma \in (0, 1]$ is a discount factor [6]. Classic RL successes include game playing and robotic control.

2.1.2. Neural Architectures

Early AI systems were built on symbolic manipulation, expert systems in the 1980s employed rule sets of the form IF condition THEN action. While interpretable, these systems struggled with ambiguity and scale. The advent of faster hardware and large labelled datasets revived neural networks in the 2010s. In computer vision, the convolutional architecture proposed by Krizhevsky et al. achieved a tenfold reduction in ImageNet classification error [7], marking the start of the deep-learning wave.

Language modelling originally used n-gram statistics; recurrent networks later captured longer context. The breakthrough came with the Transformer architecture, which employs self-attention to relate every token to every other in a single pass [8]. Scaled dot-product attention is computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right) V,$$

where $Q, K, V \in \mathbb{R}^{n \times d}$ are the query, key, and value matrices and d is the key dimension.

2.1.3. Scaling Laws and Large Language Models

Parameter counts have since grown by three orders of magnitude. Empirical scaling laws show that the loss L decreases predictably with model size N, dataset size D, and compute C following power laws such as $L \propto N^{-\alpha}$ [9]. GPT-3 (175 B parameters) demonstrated *few-shot in-context learning*: it can perform new tasks by conditioning on a handful of examples provided in the prompt, without gradient updates [10]. Such models are now labelled large language models (LLMs). The diagram in Fig. 2.1 places LLMs within the broader AI landscape.

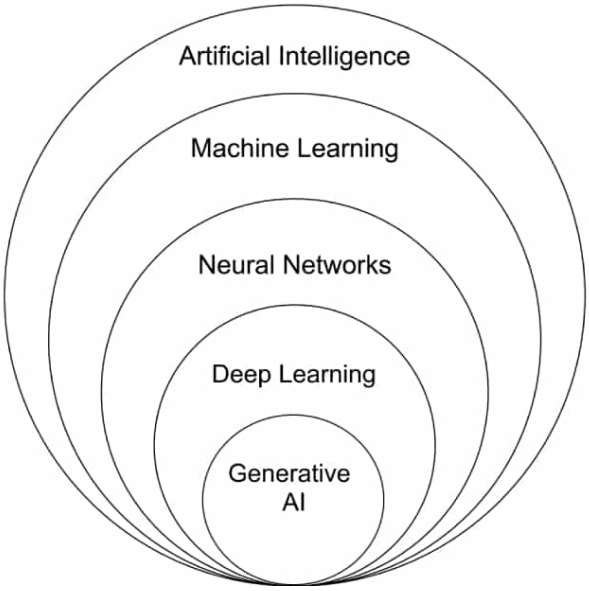

Fig. 2.1. Conceptual hierarchy often used to relate major sub-fields of AI. Generative AI (which includes LLMs) is a recent specialisation within deep learning. Reprinted from [11]

Generative AI represents a specialised subset of deep-learning techniques focused on producing new content rather than merely classifying or predicting. It encompasses image, text and audio generation through models trained to learn the joint distribution of their input data. Large language models, as shown in Figure 2.1, sit at the heart of generative AI in the text domain. By sampling from their learned distribution, they can draft articles, write code or carry on a conversation in a way that was inconceivable just a few years ago [12], [13]. These capabilities make generative AI a powerful engine for automation but also raise new challenges in reliability, safety and control.

Large language models do not operate in isolation once they leave the laboratory. In production they must retrieve e-mails, update calendars or query version-control systems. A text-only endpoint covers generation but leaves the integration burden on the host application. Each additional capability, for example sending a message or creating a task, demands bespoke glue code that is costly to maintain and difficult to secure.

Benchmarks such as APIBench report that the success rate drops by nearly thirty percent when a task requires more than one external call [14]. These findings show the distance between vetted laboratory performance and real-world usability, and lend credence to efforts to develop richer orchestration protocols, as discussed in Section 2.3.1.

Early commercial endpoints offered raw text completion. Mid-2023 updates introduced function calling: the client supplies a JSON schema, and the model emits arguments for a chosen function [15]. While practical, this interface remains stateless; the host application must stitch together multi-step workflows. Limitations of this approach motivate the tool-orchestration mechanisms discussed in Section 2.3.1.

The assistant designed in this thesis depends on three properties summarised above: (i) the ability of Transformers to handle long-range context efficiently, (ii) the empirical predictability of scaling laws, and (iii) in-context learning that allows a single model to follow tool-use instructions without additional training. Equipped with these capabilities, an LLM can move beyond text generation and act as a decision-making component within a wider software system.

2.1.4. Evolution of AI: From Symbolic to Neural Approaches

The narrative of artificial intelligence can be traced back to the 1950's with symbolic reasoning. Newell and Simon showed that machines could manipulate logical symbols to prove theorems, and expert systems in the 1980's like MYCIN encoded hundreds of "if then" rules matching human level diagnosis accuracy [3]. These systems had explanatory power, but they suffered when faced with ambiguity, or even simply the work involved in encoding knowledge for each new domain.

A transformation started when backpropagation made the training of deep neural networks practical. By 2012, AlexNet established that multilayer convolutional nets could reduce ImageNet error rates by over 90% relative to hand-designed features [7]. At about the same time, statistical learning approaches such as support vector machines and ensemble learners were based on rigorous frameworks to learn from scarce data, while neural networks excelled at learning generalisable representations from noisy examples in the real-world data.

The real turning point arrived with the Transformer in 2017. By replacing recurrence with self-attention, models gained the ability to relate every token in parallel [8]. This architecture unlocked training at unprecedented scale. When GPT-3 appeared in 2020 with 175 billion parameters and few-shot in-context learning, it revealed new emergent abilities that had not been explicitly programmed [10]. The scaling laws established that loss decreases predictably with both parameter size and dataset size [9].

The digital assistants available in modern enterprises are embodying both these approaches. They use neural architectures for pattern recognition and sequential modelling, but impose rule-based constraints to provide transparency, auditability and compliance. This allows them to have the best of both worlds - symbolic interpretability and neural flexibility, and as such, they are paving the way for hybrid systems which can reason, learn and adapt for a much wider range of business situations.

2.2. Transformer-Based Language Models

Large language models build on the Transformer by extending its capacity to handle ever-larger datasets and more diverse tasks. Originally trained with the objective of predicting the next token in a sequence, these models learn high-dimensional representations

that capture syntax, semantics and even factual knowledge [10]. Alternative pretraining schemes such as masked language modeling, where the model must reconstruct randomly masked tokens, produce complementary strengths in bidirectional understanding and are favoured by architectures like BERT [16]. Choosing between autoregressive and masked objectives depends on the downstream use case, but both approaches rely on the same core self-attention mechanism.

Positional information is essential for Transformers to respect the order of tokens. The original design used fixed sine-cosine encodings added to the input embeddings [8]. Subsequent work explored learnable positional embeddings and relative encodings that improve performance on longer texts [17]. More recently, methods like ALiBi introduce a simple bias that lets attention scores decay linearly with distance, extending the usable context window without additional parameters [18].

Scaling Transformers naively leads to quadratic growth in compute and memory with respect to sequence length. To mitigate this, several "efficient" variants have been proposed. Longformer and BigBird replace full attention in each layer with a combination of local and global attention patterns [19], [20]. The Performer uses random feature methods to approximate softmax attention in linear time [21]. These architectures allow models to process documents of tens of thousands of tokens, a crucial capability for applications requiring long-range coherence.

Another avenue for increasing capacity is mixture-of-experts (MoE). Instead of scaling a single dense network, MoE layers route each input token to a small subset of expert sub-networks. Switch Transformers and GShard demonstrated that this approach can expand model size to trillions of parameters while keeping inference costs similar to those of much smaller dense models [22], [23]. The conditional computation inherent in MoE holds promise for future assistants that must balance broad knowledge with efficient serving.

2.2.1. Autonomous Reasoning and Chain-of-Thought

In addition to expanding raw capacity, modern language models can perform multi-step reasoning when prompted appropriately. Chain-of-Thought prompting encourages the model to generate intermediate steps before arriving at a final answer, boosting accuracy on tasks involving logical inference or arithmetic by up to thirty percentage points [24]. By framing the problem as a sequence of smaller decisions, the model effectively decomposes complex tasks into manageable sub-goals.

2.2.2. Advanced Prompting Techniques

Building on Chain-of-Thought, further techniques enhance reliability and flexibility:

- Self-Consistency sampling generates multiple reasoning paths and adopts the answer that appears most frequently, reducing the impact of any single erroneous chain [25].

- Tree-of-Thoughts prompts the model to explore alternative reasoning branches in parallel, pruning unpromising paths and backtracking when needed [26].
- Retrieval-Augmented Generation combines a language model with an external knowledge base by retrieving relevant documents, appending them to the prompt, and asking the model to synthesise an answer grounded in both its parameters and the retrieved text [27].
- Prompt-Tuning and Adapter layers insert small trainable modules or soft prompts into a frozen model, enabling task-specific steering without full model fine-tuning [28].

These strategies transform a Transformer from a passive predictor into an autonomous agent capable of planning and tool use, setting the stage for the Model Context Protocol discussed in Section 2.3.1.

2.2.3. Pre-training and Fine-tuning Strategies

The remarkable capabilities of contemporary language models arise through the process of careful trainings, which generally occur in phases. Understanding these trainings is important for understanding how general-purpose models are repurposed for particular enterprise application while retaining general abilities.

Pre-training refers to the initial phase at which models learn general language understanding through huge text corpora or in-context documents. In this phase, models process billions of tokens from varied resources, including web pages, books, academic papers, and code repositories. Pre-training employs self-supervised learning due to the nature of language intent modeling, which allows unsupervised training without needing explicit labels. As the model predicts and unmasks a token or a recurrence of tokens, it learns from the surrounding content and context.

$$\mathcal{L}_{\text{pre}}(\theta) \;=\; -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

This approach enables models to absorb huge amounts of world knowledge and develop sophisticated representations of language structure, semantic relationships, and even factual information.

Pre-training scale has changed significantly in the last several years. Early language models trained on millions of training tokens; now current models pre-train on trillions of tokens. This scaling has led to emergent capabilities that tended to appear suddenly at given model sizes, which implies that scaling will likely continue to reveal new abilities in unforeseen ways. [9].

$$\text{Loss}(N, D) \approx a\, N^{-\alpha} + b\, D^{-\beta}$$

Compute requirements for pre-training have grown just as quickly. Creating state-of-the-art language models requires thousands of specialized accelerators running for months. Training GPT-3 was estimated to have cost several million dollars [10], and subsequent models have justified even higher costs. These resource requirements have made large-scale model development predominantly a space for organizations with extensive compute capabilities, though smaller models trained on specific datasets can perform well in some domains.

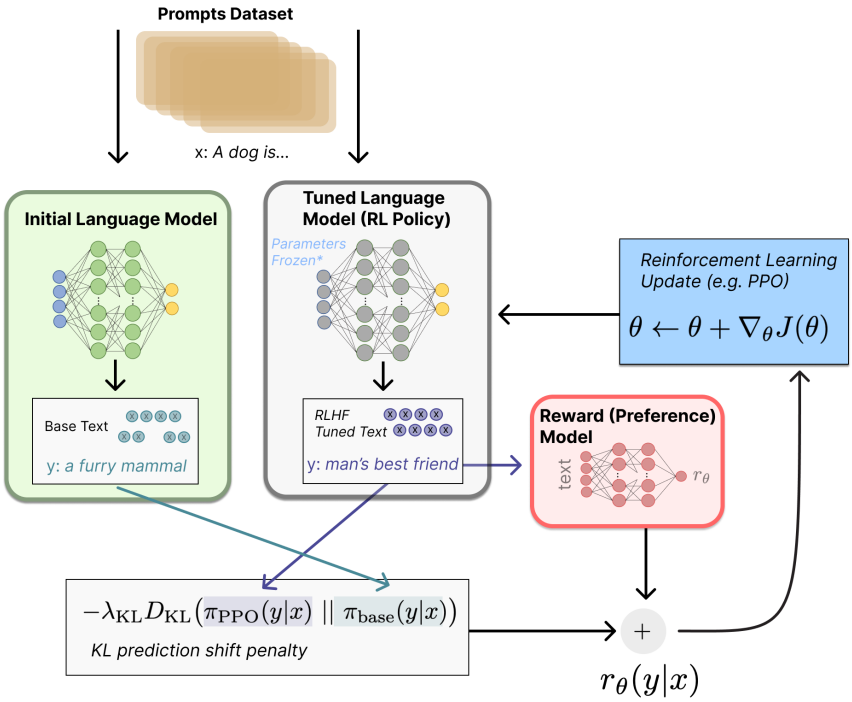

Fig. 2.2. Overview of the RLHF pipeline: from base model through human preference labeling, reward modeling and policy optimization. Reprinted from [29]

As shown in Figure 2.2, the base language model is first fine-tuned on demonstrations, then human labelers rank multiple outputs to train a reward model. Finally, a reinforcement learning update adjusts the policy to maximize this learned reward while keeping the model close to its original behavior.

Fine-tuning takes the pre-trained model and customizes it to specific tasks or domains through additional training on smaller, task specific datasets. While pre-training typically involves unsupervised learning, fine-tuning involves supervised or semi-supervised learning, which relies on labeled examples that provide desired input–output behavior. Fine-tuning is critical because it allows the models to ensure they are specialized, while also allowing them to take advantage of their general capabilities. Fine-tuning is most beneficial for enterprise applications, where models must learn how to perform specific functions, but do so with some level of consistency or reliability.

Instruction tuning is a specific and significant method of fine-tuning where models learn

to follow natural language instructions. In contrast to only learning to continue text, instruction-tuned models learn to respond appropriately to commands, questions, and requests. Instruction tuning training datasets contain many examples of instructions with appropriate responses demonstrating everything from question answering and tasks to creative writing. Instruction-tuned models are conversational assistants, not text completion engines.

Reinforcement Learning from Human Feedback (RLHF) has become an important approach for aligning language models with human preferences and values. Fine-tuning traditionally aims to optimize likelihood of target responses, although this does not predictably standardize the outputs that people might find helpful, harmless, or honest. Once RLHF involves a trained reward model on human preference comparisons [30], reinforcement learning can be used to optimize the language model against this learned reward function.

The RLHF process includes three steps. First, human annotators are asked to rank multiple model outputs to the same prompt, indicating their preference of response. Second, a reward model learns to predict human preferences from the rankings. Third, the language model is fine-tuned with policy gradient methods to maximise the expected reward over the finetuned learned preference model. This process has been critical to training language models that behave in a reasonable way in a conversational context.

As model sizes have increased, parameter-efficient fine-tuning has gained traction. Standard fine-tuning updates all model parameters, which requires a lot of computational resources and storage since we need to store one adapted model for every trained model. LoRA (Low-Rank Adaptation) and its recent variations learn only a few additional parameters respectively from the base model and leave the base model frozen. LoRA adds low-rank matrices to the model that capture the adaptations made for the task, and thus enables the user to customize the model in a way that is cheaper and uses significantly less storage.

Prefix tuning and prompt tuning are two other parameter-efficient alternatives that only modify the input representations while keeping the model parameters unchanged. These methods prepend learned embeddings to input sequences, conditioning the model's behavior without impacting its weights. This allows for fast adaptation to new tasks and also allows the same model deployment to specialize in many behaviors.

Multi-task fine-tuning exposes models to collections of related tasks at the same time and encourages the learning of shared representations that enable positive transfer across the whole task set. Multi-task fine-tuning is especially helpful for generalization when tasks share some underlying structure or require some type of similar reasoning ability. However, learning across multiple tasks effectively requires careful consideration of (1) sampling data from multiple tasks appropriately, (2) loss weighting across tasks, and (3) the dynamics of learning for the various tasks.

Domain adaptation through fine-tuning seeks to address the challenge of moving from a general-purpose model to a specific domain such as medicine, law, or finance. The domain-

specific model fine-tuning process generally involves continued pre-training on in-domain text before the model is fine-tuned on (often small) labeled examples that are task-specific. This two-stage process allows the models to learn domain-specific terminologies and knowledge while also learning how to carry out a task in that domain.

Advanced prompts can augment fine-tuning by allowing complex behaviors to occur without updating any parameters. Chain-of-Thought prompting is a type of prompt that implicitly or explicitly compel models to produce intermediate steps of reasoning before providing a final answer, which results in an important boost on complex reasoning tasks compared to the basic prompt. The products of modeling the reasoning process given sequential steps of reasoning make it possible to tackle problems that well-formed direct input–output mappings would never successfully reach.

Self-Consistency sampling generates multiple reasoning paths for any problem and chooses the most common answer, reducing the impact of reasoning errors of individual reasoning paths. It uses the model's inherent stochasticity to generate a more accurate answer without retraining.

Tree-of-Thoughts extends this principle by promoting a systematic search of possible alternative branches of reasoning and, ultimately, strategies for a more comprehensive solution to the problem.

Retrieval-Augmented Generation uses the capability of external knowledge bases augmenting static parametric knowledge. Retrieval-Augmented Generations (RAGs) retrieve the relevant documents for each query, append them to the input prompt, and ask the model to generate responses grounded in the retrieved information. It allows models to access current information and cite specific sources while maintaining their general reasoning capabilities.

Choosing between various adaptation approaches can depend on available resources, the requirements of the task, or constraints for deployment. Fine-tuning offers the best adaptations but needs a lot of resources and expertise. Parameter-efficient methods work reasonably well and may improve performance but also use far less resources. Prompting methods do not require additional training but do not optimize performance as well for specific tasks.

For an enterprise digital assistant, these three-technique combination of instruction tuning, RLHF, and more advanced prompting strategies is effective. Instruction tuning allows natural conversational interaction, RLHF makes sure proper behavior in business cases, and sophisticated prompting techniques help in accomplishing complex multi-step reasoning over enterprise data sources. This multifacet enables assistants to be reliable across diverse business scenarios and leaves the flexibility to adapt to new requirements by simply prompting alone.

The rapid evolution of training methodologies continues to expand the capabilities of language models while reducing the expertise required for effective deployment. Automated

prompt optimization, few-shot learning techniques, and improved parameter-efficient methods are making sophisticated AI capabilities accessible to organizations without extensive machine learning expertise. These developments are crucial for enabling widespread adoption of AI assistants in enterprise environments.

2.3. Large Language Models and Tool Use

This section reviews how large language models have evolved from simple text-only systems to agents capable of invoking external tools. We first trace the move from early 'function calling' APIs to the Model Context Protocol (MCP). We then examine current interoperability standards that aim to make tool integration model-agnostic

2.3.1. From Function Calling to MCP

The first generation of tool-aware language-model APIs appeared in mid-2023. Developers would list a set of functions in JSON Schema, include the schema in the prompt, and let the model reply with a single JSON object that matched one of the definitions. Research prototypes such as Gorilla showed that a finetuned model could fill in the correct arguments for more than 1 600 public APIs, yet every call remained stateless and the model forgot past tool invocations once the response was returned [14]. A parallel line of work framed prompting itself as a kind of "ad-hoc programming language" able to trigger calculators or search engines but again with no persistent context [31].

As projects grew, two pain-points became evident. First, every combination of model (N) and tool (M) required bespoke glue code, leading to the well-known $N \times M$ integration problem [32]. Second, the one-shot request-response style prevented multi-step workflows: a model could draft an e-mail but could not remember that it had already called the calendar API two turns earlier. Large-scale benchmarks such as APIBench confirmed that success rates dropped sharply once more than one call was needed [14].

The Model Context Protocol (MCP), open-sourced in late 2024, tackles both issues through a client–server design. The protocol keeps a bidirectional channel where messages, tool calls, and intermediate results share the same conversation state. Tools are advertised by an MCP server, discovered dynamically by the client, and invoked through JSON-RPC without extra adapters [1], [2]. Because the context object is persisted, the agent can chain calls—read an e-mail, fetch a spreadsheet, update a ticket—while reasoning over the cumulative state.

Early case studies report sizeable engineering gains: one industrial deployment cut integration code by 68 % and reduced average latency by 35 % compared with a function-calling baseline [32]. The shared context also simplifies auditing; security work now focuses on protocol-level safeguards rather than per-tool wrappers, exemplified by the MCPSafetyScanner introduced by Radosevich and Halloran [33]. In practice, MCP makes it feasible

for a single agent to orchestrate many connectors in production, which is precisely the capability required for the assistant described in this project.

2.3.2. Interoperability Standards

As tool-using language models multiplied, several communities began to define open standards aimed at simplifying integration and reducing vendor lock-in. The guiding idea is to separate the "how" of a tool call, transport, authentication, error handling, from the "what" that the model wants to achieve. This paragraph surveys three initiatives that frame the current discussion and explains how MCP fits among them.

Originally devised for code editors, LSP introduced a JSON-RPC channel in which an IDE and a compiler process exchange structured messages [34]. Although LSP was not designed for large language models, it demonstrated that a generic request–response envelope can survive across programming languages and tooling stacks. Recent experiments have adapted LSP to serve as a lightweight bridge between an LLM and debugging utilities, but the protocol lacks native concepts such as tool discovery or role-based authorisation that modern AI agents require.

Some providers responded by embedding OpenAPI documents directly into prompts. The model receives endpoint descriptions and returns a JSON stub that the host application must execute. While this approach is easy to prototype, it remains tightly coupled to one vendor's runtime and keeps the session stateless. Comparative benchmarks report a drop of almost thirty per cent in success rate when more than two calls are chained, largely because the model has no persistent store for intermediate results.

The Model Context Protocol extends the JSON-RPC idea with three missing features: dynamic service discovery, long-lived context objects, and standardised error recovery [1], [2]. Because tools register themselves with an MCP server, the same client can call new services at runtime without schema edits. Radosevich and Halloran show that a security wrapper placed at protocol level blocks seventy-five per cent of injection attempts that bypass function-calling defences [33]. Early adopters therefore view MCP not as a competitor to OpenAPI or LSP but as a higher-level conduit that reuses JSON-RPC while adding conversation state and role isolation.

In summary, interoperability standards have moved from IDE-centric protocols through prompt-embedded schemas to the stateful channels exemplified by MCP. The project described in this thesis adopts MCP because it offers the persistent context and multi-tool orchestration that a start-up workflow demands, while remaining compatible with existing JSON-based service descriptions.

2.4. Enterprise Digital Assistants

Enterprise Digital Assistants Enterprise Digital Assistants (EDAs) are AI-enabled agents designed for corporate use-case scenarios as a single natural-language interface to an organization's data and services .

For example, EDAs interface with calendars, document repositories, ticketing systems and communication systems by using either APIs and/or MCPs. They can automate many routine tasks, such as preparing a meeting agenda, retrieving documents, and tracking project status [2], [14].

Their design is to allow the applications to be used by hundreds or thousands of other users (concurrently) and ensure authentication, authorization, and compliance controls are in place to enforce corporate governance over proprietary data used by the EDA [32].

In the last decade, EDAs have developed in lockstep with the development of largescale language models and integration frameworks: what started as relatively simple rule-based helpdesk bot applications are now +mixed-initiative systems + that integrate neural language comprehension with workflow orchestration tools. Modern EDAs often monitor processes proactively and suggest reassignment of tasks in an effort to resolve reported bottlenecks, which helps teams continue needed work that is not requiring human intervention [3], [35]. In the following sections we review leading Commercial Solutions that are available on the market, evaluate their Productivity Impact via quantitative studies, and look at their level of Autonomy and the role of humans in the loop.

2.4.1. Commercial Solutions

Commercial EDAs today include consumer-derived assistants adapted for business and specialized AI companions embedded in collaboration platforms. For example, Apple's Siri supports corporate workflows through SiriKit and the App Intents framework, allowing enterprise apps to expose actions such as scheduling meetings, sending messages or retrieving files from cloud storage [36]. Users invoke Siri with voice or typed commands on iOS, macOS and watchOS devices, which makes it easy to automate common tasks without leaving their primary workspace.

GitHub and Microsoft have also entered the EDA market with Copilot. GitHub Copilot delivers code completion and a conversational chat interface directly within popular Integrated Development Environments (IDEs) such as Visual Studio Code, JetBrains IDEs and Eclipse, and it surfaces contextual insights by connecting to GitHub issues, pull requests and repository documentation [37]. Enterprise administrators can enable Copilot organization-wide, enforce usage policies and even use Microsoft 365 Copilot connectors to index content from GitHub repositories for richer context in their workflows.

In the collaboration domain, Salesforce unveiled Slack GPT as a native generative AI assistant inside Slack [38]. Slack GPT comes with prebuilt AI app integrations, supports

multiple language models and taps securely into Customer 360 and Data Cloud. Beyond the native offering, third-party SlackGPT apps on the Slack Marketplace provide connectors to services such as Google Drive, Notion and GitHub, enabling teams to generate summaries, answer questions about project files and automate routine processes directly within Slack channels [39].

Assistant	Connectors	Proactivity	Cost (per user/mo)	Deployment Requirements
Google Duet AI	Gmail, Docs, Sheets, Slides, Drive, Meet	Proactive summaries and suggestions	$20–$30	Google Workspace Enterprise subscription
Microsoft 365 Copilot	Word, Excel, PowerPoint, Outlook, Teams, OneDrive, SharePoint	Proactive insights and alerts	$30	Microsoft 365 E3/E5 license with Copilot add-on
Asana AI	Tasks, Projects, Custom Fields, Forms, Timeline, Work Graph	Reactive suggestions and auto-subtasks	$24.99	Asana Business Plan subscription

TABLE 2.1. COMPARISON OF POPULAR DIGITAL ASSISTANTS WITH ENTERPRISE EXTENSIONS.

Table 2.1 highlights how different assistants connect to corporate systems, their level of initiative and the rough cost per user. The proactivity column establishes whether an assistant responds to commands only or if proactive response actions are taken, as guided by the context. The cost and deployment requirement columns define vendor-published costs and consider any infrastructure or licensing requirements for an assistant to be deployed in an enterprise context.

2.4.2. Productivity Impact

In the last few years the effect of enterprise digital assistants on work productivity has been quite noticeable. Studies show that when assistants work well they save users time on routine tasks and help them focus on higher-value activities. For example, large surveys link satisfaction with an assistant to measurable improvements in individual output, and field experiments report double-digit gains in task throughput when AI support is available at scale [40], [41].

Such quantitative evidence shows that assistants not only reduce manual effort but also enhance engagement by providing timely guidance and feedback. In the upcoming paragraphs we present four studies with empirical evidence for time savings, engagement improvements, and the contexts in which assistants provide the most value.

Alexa, Let's Talk About My Productivity (Marikyan et al. 2022). In a survey of 536 professionals who use an AI-based digital assistant at work, the researchers identified five main drivers of user satisfaction: expectation of performance, perceived enjoyment, attribution of intelligence, social presence, and trust. Greater user expectations were associated with higher satisfaction, which in turn was associated with significantly higher self-reported productivity and work engagement. Participants reported that on average the

assistant saved them 12 percent of their weekly work time, primarily through automated meeting scheduling and document retrieval. [40].

This finding shows how a little saved time on regular jobs can multiply into larger efficiency gains. In an engineering context, freeing up a few hours per week can mean more design iterations or testing or learning a new tool, which all contribute to the advancement of project delivery and innovation.

Partnering with AI (Cranefield et al. 2022). This article, based on intensive interviews with a sample of 28 users, investigates how digital productivity assistants influence work habits. The productivity assistants supplied customized analytics, reminders, and recommendations that helped the users co-regulate their work. But, a range of issues prevented users from enjoying the full level of benefits, based on their perceptions of the assistant's accuracy, feedback transparency, and configurability. The authors argue that user control and clarity of operation are key drivers of user perceived productivity gains. [42].

A Review of Digital Assistants in Production and Logistics (Zheng et al. 2024). This systematic literature review outlines assistants used in manufacturing and supply-chain environments. Across different scenarios from quality control to maintenance checklists and the workers reported completion times to be as much as 20 percent faster with assistants providing a combined audio and visual cue. While most instances relied on some form of human oversight, multisensory guidance supported the difficult cognitive work associated with complex procedures, resulting in increased intuitiveness while reducing errors by as much as 15 percent [43].

Generative AI at Work (Brynjolfsson et al. 2025). When introducing a conversational AI assistant in a staggered trial to 5 000 customer-service staff, the availability of the assistant boosted hourly cases solved by 14–15 percent on average. Less experienced staff benefited most, and veterans maintained high quality with modest speed gains. The research also observed a minor increase in customer satisfaction when using AI support [41].

The findings emphasize how generative AI can act as a force multiplier in large teams. Staff members who are new to the organization will feel more empowered to get queries resolved faster and with greater confidence, helping to flatten learning curves and raise overall service standards. This suggests a future in which generative AI not only frees up time to automate tasks but also accelerates skill development.

Overall, these studies suggest that digital assistants can offer significant time savings and engagement gains across office, manufacturing and service situations. Similarly, based on this evidence, our research is extending the capabilities of the assistants into other enterprise domains with higher levels of automation. By infusing more extensive context protocols and taking a more proactive planning approach, we hope to help teams deal with complex workflows, and drive even higher productivity gains.

In the end, the power of AI-enabled assistants is more about reimagining the employee relationship to organizational knowledge, not just abbreviating the execution of routine

tasks. In the following chapters we detail our system design and implementation that will allow for as much autonomy as the design can reasonably incorporate for successful collaboration and cohesion between AI agents, and human teams.

2.4.3. Autonomy and Human-in-the-Loop

When creating enterprise digital assistants engineers try to provide as much autonomy as is appropriate for it to perform its duties, while still ensuring that humans are "in control". For example, an assistant may manage its own workflow through automatically organizing incoming requests or drafting status reports without waiting for an input. This kind of autonomy reduces workload but it does not remove the need for human judgement in complex or unclear situations.

The idea of human in the loop underlines why people must oversee critical decisions. In their study “Human in the loop or AI in the loop Automate or Collaborate” Natarajan and colleagues point out that many systems labelled as interactive assistants really follow an AI in the loop model where the machine performs most actions but relies on a human to guide or correct its outputs [44]. This view reminds us that the user remains at the centre of every workflow and that the assistant is first and foremost a support tool.

That shift in perspective has real consequences for reliability and safety. If developers assume that an assistant will act perfectly when set to full autonomy they might skip vital validation steps and cause errors in email handling or scheduling. According to Natarajan et al. it is essential that humans review suggested actions before they take effect in sensitive areas such as finance or compliance.

Effective designs therefore incorporate clear checkpoints where the assistant explains its reasoning and waits for human confirmation. For instance, a task planner could propose a new project timeline then display the main assumptions so that a manager can approve or adjust the plan. By including simple approval interfaces engineers help users build trust while reducing the effort of continuous oversight.

Achieving the right level of autonomy balanced with human intervention requires careful adjustment. Too little autonomy can lead to frustration for users who are called upon to intervene on trivial things. Too much autonomy can lead to costly errors if the system misreads context or lacks domain knowledge. The ability to allow teams to adjust the level of autonomy makes it possible for the same assistant to serve different roles and experience levels across the company.

In the end these digital assistants serve to amplify human capabilities rather than replace them. They can manage routine or data intensive operations but they still depend on human creativity ethical judgement and strategic insight. Designing enterprise assistants with robust human in the loop mechanisms ensures that automation respects company policies adapts over time and supports employees in making well informed decisions [44].

2.5. Task Planning and Optimization Approaches

Efficient task scheduling and optimization is essential for any AI assistant operating in a busy enterprise context. Scheduling work items in the right order, managing resources wisely and minimizing gaps creates the potential for many projects to proceed smoothly and liberates human users from routine co-ordination. In our assistant, which connects calendars, document stores and external services, effective planning restrains clashes, balances workloads and reduces the need for manual follow-up. Section 2.5 describes the algorithms and reinforcement-learning methods underpinning these scheduling decisions, and how they provide the ability for the assistant to adapt dynamically as demands evolve.

2.5.1. Prompt-Based Reasoning (CoT and ToT)

Prompt-based reasoning has emerged as a key mechanism of modern language model capabilities. A large language model can now solve complex, multifaceted problems by following a structured prompts that include intermediate steps. This process provides benefits for both correctness and transparency, as the model illustrates its thought process rather than providing a final answer.

Chain of Thought prompting encouarages the model to treat the problem as a series of too questions. Each sub-question follows naturally from the last, bringing the model a step closer to a correct answer. For example, when solving a math problem, the model lays out each step of the calculation in order. This process was first examined in depth by Wei and coworkers and showed gains for reasoning tasks [24]. When using the Chain of Thought prompt, users can see the steps taken by the model, which allows them to check the model's logic and catch errors in the thought process early on.

Tree of Thoughts takes this concept one step further and has a model run through many reasoning paths in parallel. Instead of exploring one line of reasoning, the model branches out, assesses a few paths, and returns to a single line of thinking with the best known solution. Research teams at MIT and IBM have shown the branching method found better answers to puzzles and planning tasks rather than a linear prompt [26]. A simple graphic that highlights a single line of thought compared with a tree of across many possibilities, clearly delineates this difference, and it helps explain why ToT's approach could outperform CoT in difficult tasks.

2.5.2. Hybrid LLM + External Solvers

A combination of a language model and an accurate external solver is the ultimate combination. The LLM understands context, respects user instruction and translates through natural language input for the user. The Solver will help optimize with mathematical rigor and accuracy, something a language models can be less robust with.

Under this hybrid structure, the workflow consists of three steps. The first step is that the LLM takes a user specification and converts it into a formal problem statement, commonly expressed in a formal language like linear programming (LP) or mixed–integer programming (MIP). The second step is that an external solver (for example, the MIT LP–solver framework) takes the specification and finds an optimal solution. The third step is that the LLM takes the output from the solver, parses it and communicates it back to the user in clear prose that is user–friendly. Szeider's work with the MCP-Solver, demonstrates how this pattern can empirically reduce scheduling errors by 50% due to the balance of LLM flexibility and CP solver accuracy. [1].

This technique is powerful in places like logistics planning, where a user will say "find the most efficient delivery routes for tonight's orders." The LLM curates the routing problem, the solver returns the minimum cost paths, and the assistant wraps it all up by explaining that ("Deliver to A then B then C for total travel time of X hours"). By relying on the solver to handle the heavy-duty optimization, the system could evaluate hundreds of locations with formal guarantees while still offering a conversational interface that end users would expect.

However it comes with challenges. As noted previously, the process of formally encoding human language into precise mathematical constraints takes careful prompt engineering. Guarantees of end–to–end correctness mean verifying both solver input and output. Caching or asynchronous calls are often necessary to reduce solver response latency when problems become larger.

2.5.3. Reinforcement-Learning-Based Scheduling

Scheduling and planning are good fit for reinforcement learning because they require sequential decision-making in shifting environments. To this end, an RL agent is trained to take actions to maximize a reward in the long-term; instead of just taking the best immediate action. As such, it is the natural choice for workflows that last hours or days while balancing multiple objectives (e.g. throughput, fairness and cost).

In RL scheduling the agent observes the current state of a system and selects an action, like assigning a job to a machine, that leads to a reward signal on performance measures. Over many episodes the agent adjusts its policy to make better decisions. The key ingredients of RL are the state space, action set, reward function and policy. The policy uses states to map to actions to maximize expected cumulative rewards.

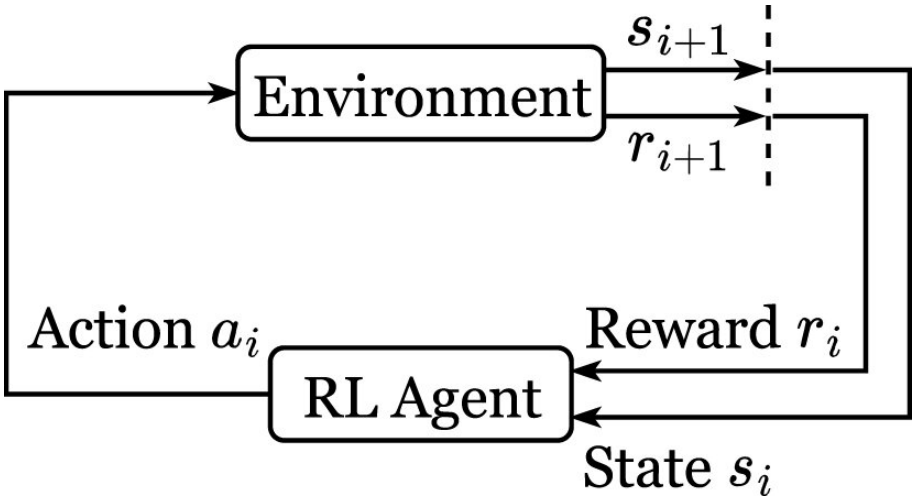

Fig. 2.3. Fundamental diagram of the Reinforcement Learning (RL) loop, showing the cyclical interaction between the agent and the environment for sequential decision-making. Reprinted from [45]

Taxonomy and Surveys

Surveys and taxonomies play a crucial role in mapping out the diverse approaches to RL scheduling. They classify methods by learning algorithm, state representation and reward design.

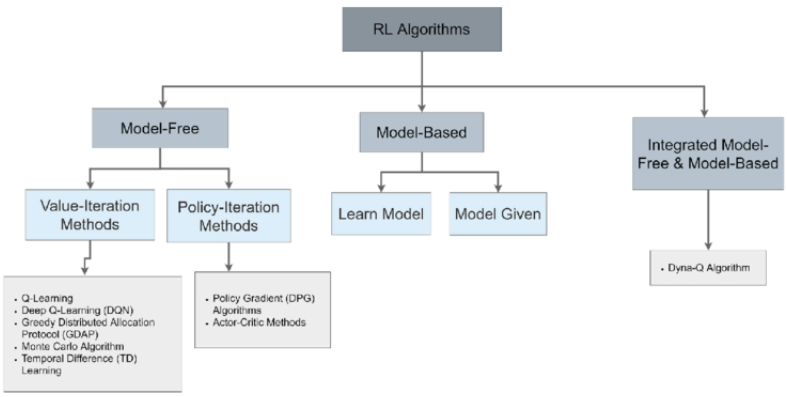

Fig. 2.4. Taxonomy of the main Reinforcement Learning algorithms, categorized by their approach (value-based, policy-based, actor-critic, model-based, and meta-RL) . Reprinted from [46]

Jayanetti and others provide one of the most thorough reviews, highlighting distinctions between value-based, policy-based and actor-critic methods [47]. Beck et al. provide a complete survey of meta-reinforcement learning approaches, including multi-objective formulations, and explain how they used these methods for task-scheduling problems [48]. These reviews improve direction for researchers with possible gaps needing attention.

Cloud Workflow RL

Using RL, however, in cloud environments, can present many challenges. We have an ever-changing workload, and hardware is heterogeneous (and even the same hardware may exhibit different performance profiles over time). A RL agent, when scheduling jobs to a virtual machine, must learn how to balance cost, latency and energy when scheduling jobs.

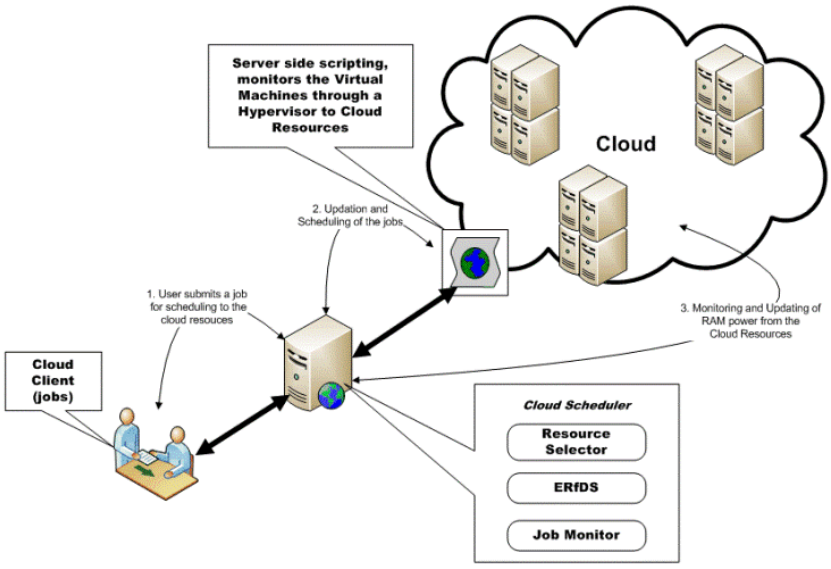

Fig. 2.5. Diagram of a workflow scheduling system in cloud environments based on Reinforcement Learning, illustrating dynamic resource management to optimize key metrics. Reprinted from [49]

Shen et al. enhance cloud workflow scheduling under dynamic demand by incorporating an evolutionary reinforcement-learning loop with self-attention mechanisms. Their approach has a lower average latency of about 15 percent compared to static heuristics, while keeping costs low [50].

Interactive LLM Agents

Recent work evaluates systems where an LLM (language model) is constructed as a planning layer on top of an RL (reinforcement learning) agent. The language model interprets user commands, creates high-level objectives, and transmits them to the RL scheduler. In this latter work, Dalal et al. introduce Plan-Seq-Learn, a framework (Plan-Seq-Learn-architecture) where an LLM does two things: it reads the high-level user command and translates this into subgoals for the RL agent; and then in turn, iteratively refines and optimizes long-horizon plans for the LLM agent based on natural-language feedback in its environment. While the focus applies to robotic tasks, the same precedent applies directly to intelligent scheduling [51]. A diagram showing the LLM guiding the RL loop helps to visualise how human directives flow through the system.

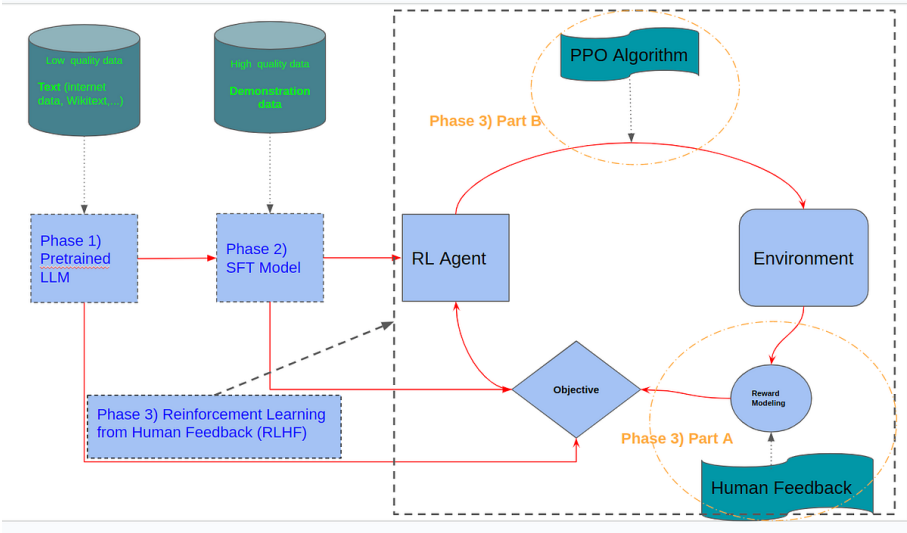

Fig. 2.6. Interaction between a Large Language Model (LLM) and a Reinforcement Learning agent, where the LLM serves as a planning layer and enables natural language feedback from the user. Reprinted from [52]

Cost-Aware RL

In cost-aware RL the reward function explicitly penalises expensive actions. For example an agent might receive reward $R_t = U_t - \lambda C_t$, where U_t is utility at time t, C_t is the incurred cost and λ weights cost sensitivity [53]. This design encourages the agent to look for efficient solutions that meet performance targets without overspending. Tuning the parameter λ becomes essential to balance cost and quality.

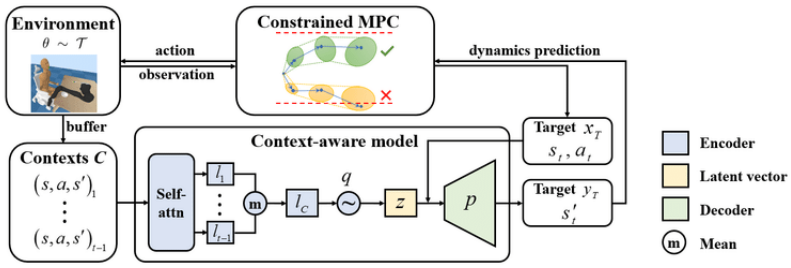

Fig. 2.7. Conceptual representation of the trade-off between performance and cost in cost-aware Reinforcement Learning systems, showing the Pareto frontier of optimal solutions. Reprinted from [54]

2.5.4. Multi-Agent Collaboration

When tasks in a process become larger and more complex than one agent can comprehensively manage, multi-agent collaboration allows several AI agents to work in concert to share the workload. Agents can specialize in subtasks to work together to achieve a common goal.

This method offers robustness and scalability. In situations like distributing scheduling, one agent could be concerned with resource availability and another agent on task prioritization. Projects like AutoGPT [55] and BabyAGI [56] show how simple agent loops can be constructed to become more comprehensive implementations. Users have reported that these frameworks allow them to create complex workflows easily and without creating a bottleneck central point.

Agents coordinate by either sharing state information or sending messages through a shared bus. Choosing the communication layer creates problems such as preventing conflicts when two agents both request the same resource. The need to converge towards a coherent plan often requires the assistance of negotiation protocols, or some shared reward signal. These elements create multi-agent systems that are both powerful and challenging to develop.

2.6. Synthesis and Research Gaps

The current state of artificial intelligence, and digital assistants in particular, has exhibited remarkable and rapid progress over the last few years. As described in earlier sections, developments in large language models, newer neural architectures and novel learning paradigms provide the foundation for highly integrated systems that can understand and generate human language with extraordinary fluency. We have identified the progression from early rule-based systems to deep-learning and reinforcement-learning agents. A key development in this domain has been the emergence of MCPs. These architectures integrate many capabilities together, along with disparate sources of data, into a single seamless solution that did not exist before. MCPs demonstrate through layers the integration of language understanding, integrations with structured APIs, and proactive planning, and therefore provide natural engagement and large breadth of potential use cases in the enterprise space.

MCPs present an opportunity to integrate services for email, project management such as Trello and GitHub, calendar systems using Google Calendar, and cloud storage, like Google Drive. Instead of an unwieldy platform model or combining APIs together, a multi-modal agent can serve as a nerve centre and act on complicated workflows, to collect information and automatically adapt in use to become relevant. While improving things is one level of orchestration, that expanded use may enable you to address issues that required excessive human coordination previously. Future digital assistants will thrive at this junction of intelligence and connectivity, or such examples will emerge.

Despite the advances in the individual components and MCPs, there has not been much research that has even looked at a pro-active and personalized autonomous assistant that can integrate together many enterprise tools. There are some studies that focus on one aspect of optimizing a workflow in a narrow sense, and there have been surveys on meta-reinforcement learning, but none have addressed our vision of an assistant that proactively plans daily activities based on the user's strengths while discovering where they could have

been more effective across various management and communication channels. Therefore, the gap in the literature for this research project to address is: how to truly realize the potential of be empowering and an intelligent assistant in enterprise situations.

The source of this research gap is the fundamental difficulty of integrating and orchestrating a set of functionalities, all having unique interaction models and technical requirements. Most efforts so far have focused on optimizing a single aspect of assistant behavior, like natural language understanding, or connecting (at most) one or two external services. Few works have sought to create a holistic system that acts decisively as an "operational brain" for an entire organization. The challenges for building a persistent, operational decision support system that can offer personalized, proactive recommendations in addition to executing commands presents substantial algorithmic and architectural challenges. Creating a set of models that can monitor the entire context of an enterprise and be designed to incorporate decisions based on dozens of highly heterogeneous data streams is a daunting design and computational problem. Also, it is important to remark that all of this is all a very recent field, and as it has alreaedy been mentioned the apperance of the tool that made this possible, the MCPs , was in late 2024. So, the recency of all of this explains too why proper research and mature tools have not been developed.

Today's enterprise assistants are impressive for their narrow use cases, but they lack the breadth and holistic support organized activity needs. They are typically a collection of fragmented tools designed to perform individual tasks, such as managing email, tracking projects or scheduling events, but there is little to no interconnectedness. As a result, they cannot meaningfully optimize workflows across platforms, identify inefficiencies across platforms, or proactively and/or personally aid individuals in the workplace. For example, no assistant today can connect a user's Trello workload, compare it to a calendar created with Google Calendar, correlate it with their GitHub activity, and provide a user with the best work plan for a day by assembling it into a calendar. The use case is contractually constrained to acting on commands issued when invoked without contextual intelligence or cross-platform comparison to maximize individual productivity and organizational effectiveness.

At Missio IA, we have identified this compelling need and feel the next frontier for enterprise assistance involves developing truly intelligent and proactive systems. Motivated by curiosity and leading edge ambition, we began the process of developing a digital agent to address the research gaps we have identified above. We propose an MCP-based assistant that is not just task automation, it will also be a workflow optimizer, performance assessors, and facilitators of interactions.

The assistant at the heart of this thesis has many original capabilities that distinguish it from current solutions. By integrating with multiple MCPs , it receives the ability :

- Communication and Client Management. The assistant follows received email messages continuously, uses Google Search and Trello data, and then categorizes the sender as either client or prospect and the inquiry as complaint, bug report or

new business offer. All of this is captured automatically in dedicated Google Drive documents. The assistant even suggests tailored offers for quote requests.

- Proactive Workflow Optimization. Every morning at 4 a.m., the agent reviews all open Trello cards, GitHub repo activity, and every team member's Google Calendar. Considering their strengths and abilities, it creates a personalized and optimized work plan for the day. The plan is also captured in Drive, including an explanation for each suggestion.

- Inefficiency Detection and Reporting. The assistant autonomously identifies workflow stoppages, stalled tasks, or resource misallocations, generating a detailed report for review and action.

- Personalized Consultation Bot. The staff can ask questions to an advanced chatbot that has full access to all the data of the company, that is collected by MCPs . This enables the chatbot to answer complicated queries at the company level and by using connected services, take direct action.

- Meeting Automation and Relationship Management. The agent retrieves Google Meet transcripts and identifies the client company, saves the transcript into that company's Drive folder, summarizes the meeting, and highlights the significant points and opportunities for improvement. It collects personal details (eg interests or family details) and inserts them into a private folder, allowing for a more personal and empathetic experience when working with clients and collaborators for everyone in the company.

These features, as detailed in section 5, show the assistant's ability to operate effectively and deliver significant task-optimization and automation support within Missio IA.

Functionality	Google Duet AI	Microsoft 365 Copilot	Asana AI	Missio OS
Email Management	Compose and summarize in Gmail [3]	Draft and recap in Outlook [4]	No	Classification, logging, offer proposals
Project Tracking (Trello)	No	No	Native task management in Asana [5]	Daily analysis, task assignment, blockage detection
Repository Management (GitHub)	No	Code assistance in IDEs, but no repo analytics	No	Progress analysis, workflow optimization
Calendar Management	No (Meet note-taking only) [6]	Limited event summaries in Teams/Outlook [7]	No	Proactive analysis, personalized planning
File Management (Drive/OneDrive)	Summaries and action-items in Drive [8]	Summarize and cite in SharePoint/OneDrive [9]	No	Automatic logs, report generation, structured storage
Content/Proposal Generation	Full drafts in Docs, Slides [10]	Drafts in Word, PPT, Excel [11]	AI-powered task descriptions [12]	Customized offer proposals
Personalized Daily Planning	No	No	No	Yes (strength-based, workload-aware)
Blockage and Inefficiency Detection	No	No	No	Yes (cross-platform, proactive reporting)
Advanced Consultation Chatbot	Gemini in Chat, Side Panel [13]	Copilot Chat with memory [14]	Asana Intelligence chatbot [15]	Full MCP access, complex queries
Meeting Transcript Analysis	Auto notes in Meet [16]	Intelligent meeting recaps in Teams [17]	No	Summaries, client insights, private notes
Relationship Data Management	No	No	No	Yes (personal details for empathetic interaction)
Autonomy and Proactivity	Moderate (summaries and suggestions)	Moderate (alerts and recaps)	Moderate (task suggestions)	High (daily autonomous operations)
Holistic Enterprise View	Partial (Workspace only)	Partial (M365 apps)	Partial (Asana data)	Full (MCPs to access all company's data integrated)

TABLE 2.2. COMPARISON OF ENTERPRISE DIGITAL ASSISTANTS AND MISSIO OS

As shown in Table 2.2, existing products in the market offer integrated well-developed features in certain areas to solve specific pain points, however, none offer the system integration and holistic approach that Missio OS does. The capability for our assistant to autonomously present an experience across platforms, knowledge of forthcoming user needs, customize workflows, and maintain sensitive relationship information is only a few steps ahead of where existing digital assistants in the enterprise are today. This work not only demonstrates that a system design of this nature is feasible, but it also lays the groundwork for a new foundation of what AI in corporate environments can look like.

3. SYSTEM REQUIREMENTS AND DESIGN

This chapter will detail the requirements that shaped the development of Missio OS, describing both of its functional capabilities and the non-functional qualities for a resilient system. Lastly it will describe the systems architectural overview, the technology stack, and considerations with regards security and privacy.

3.1. Functional Requirements

These are the capabilities and behaviors that Missio OS must exhibit in order to effectively assist and automate operations within Missio IA. Each requirement is stated clearly and is meant to be verifiable.

3.1.1. Client Communication and Information Management

FR-001: Email Processing and Meeting Transcripts Processing.

The system shall be able to access and process all incoming emails from all sources, specially clients or potential clients, and do the corresponding action depending on the content of the email.

- **FR-001.1: Gmail Integration.** The system shall access the mailing system used by the company. In this case, it is Gmail. It needs to be able to access it in order to fulfil the rest of requirements
- **FR-001.2: Google Drive Integration.** The system shall access the company's Google Drive. It shall create documents, delete them, read them and modify them, and navigate through the folders structure, or even create new folders
- **FR-001.3: Sender Identification and Classification.** The system shall identify and classify email senders as existing clients, potential clients, or other sources, based on internal company data and email context.
- **FR-001.4: Email Content Categorization.** The system shall categorize the content of incoming emails into predefined types, such as complaint, error report, offer inquiry, general query, or spam.
- **FR-001.5: Google Drive Document Creation.** The system shall automatically create and store relevant documents, or summaries or conclusions, for each processed email in the corresponding Google Drive company folder, so that it can be later accessed by the system or a member of the company.

- **FR-001.6: Draft Generation and registration.** The system shall generate draft replies or content for emails based on user previous email responses, so it must be able to access all the previous user emails. All the drafts generated have to be registered in the corresponding Google Drive company folder.
- **FR-001.7: Offer Proposal Generation.** For emails identified as offer inquiries, the system shall generate several product proposals tailored to the inquiry's details, so that they can later be analysed by a team member, who will start working from the best one.

FR-002: Meeting Transcription Analysis and Relationship Management.

The system shall process and derive insights from meeting transcriptions to enhance client interactions.

- **FR-002.1: Transcript Ingestion and Storage.** The system shall accept and store meeting transcriptions in the corresponding Google Drive company folder.
- **FR-002.2: Business Entity Profile (BEP) Identification.** The system shall recognise which client or possible client the meeting was with, to store the documents generated in the corresponding Google Drive company folder, or if it is a new client it shall notice and create the corresponding Google Drive company folder to store the documents there.
- **FR-002.3: Meeting Summary and Analysis.** The system shall automatically create and store or summaries, and analysis of what the team did right or wrong, improvement advice for the team, and an analysis of what could be done to solve the client requests of the meeting, in the corresponding Google Drive company folder.
- **FR-002.4: Private Relationship Data Extraction.** The system shall extract and store personal information given by the other company members in the meeting. These would be stored and once a company member has a meeting with that company again, the system shall inform the team member, so that he can be more empathic and generate a better relationship with the client staff.

3.1.2. Internal Workflow Automation and Optimization

FR-003: Daily Personalized Work Planning.

The system shall autonomously generate an optimized, personalized work plan for each team member on a daily basis.

- **FR-003.1: Trello Task Integration.** The system shall access all the boards of the company's Trello workspace. It shall create new tasks, modify them, and read them.

It shall scan all the company tasks and determine wether their development is being optimal or not.

- **FR-003.2: GitHub Progress Monitoring.** The system shall integrate and analyze progress data from team members' GitHub repositories, by consulting commits, pull requests or issues resolution.
- **FR-003.3: Google Calendar Integration** The system shall access all the individual calendars, to check the events and keep them in mind in the daily planning process
- **FR-003.4: Strength-Based Task Allocation.** The system shall have access to a technical sheet where the strengths of each team member are exposed, and keep them in mind while distributing tasks and being more or less demanding with a team member
- **FR-003.5: Plan Generation.** The system shall provide a clear explanation and justification for each task allocation and prioritization in the personalized daily plan.
- **FR-003.6: Google Drive Plan Storage.** All generated personalized daily plans shall be stored in the corresponding Google Drive company folder.

FR-004: Blockage and Inefficiency Detection.

The system shall continuously monitor integrated platforms to identify and report workflow impediments and suboptimal processes.

- **FR-004.1: Cross-Platform Monitoring.** The system shall analyze data from Trello, GitHub, Google Calendar and Google Drive files to detect patterns indicative of blockages, delays, or inefficient resource utilization.
- **FR-004.2: Progress verification.** The system shall compare the extracted conclusions of a day's progress and blockages with the conclusions of the previous days, to be able to notice if a problem is not progressing even if being implemented a solution, and propose a different solution or approach.
- **FR-004.3: Automated Reporting.** The system shall generate structured reports detailing identified blockages and inefficiencies, including potential causes and affected areas, and giving solutions for the detected problems.
- **FR-004.4: Team Notification.** The system shall alert team members via email of the detected blockages, in order to have them solved as soon as possible.

3.1.3. Intelligent Assistant and User Interaction

FR-005: Advanced Conversational Capabilities.

The system shall provide a sophisticated chatbot interface capable of understanding and responding to complex natural language queries, and accessing all the information and organization sources of the company.

- **FR-005.1: Cross-MCP Information Retrieval.** The chatbot shall be able to retrieve and synthesize information from all integrated Model Context Protocols (Mail, GitHub, Trello, Google Calendar, Firecrawl, Google Drive), and combine their actions in response to user queries.
- **FR-005.2: Dynamic Action Execution.** The system shall allow users to initiate a wide range of actions across the integrated platforms through conversational commands. These actions include:
  - **FR-005.2.1: Task Management.** Creating, assigning, updating, or querying tasks in Trello.
  - **FR-005.2.2: Code Management.** Querying the status of a GitHub repository, creating issues, or performing other basic code-based interactions, such as creating a new branch. Also changing code directly in the github repository and pushing the changes, as the user requests.
  - **FR-005.2.3: Information Search.** Performing targeted searches within Google Drive or conducting general internet searches or deep research processeses to find the information that the user needs.
  - **FR-005.2.4: Email Communication.** Drafting, analysing, sending or archiving mails.
  - **FR-005.2.5: Data Retrieval.** Accessing specific documents or reports from Google Drive.
  - **FR-005.2.6: Event Handling.** Changing, creating, canceling and inviting people to calendar events. They might include a Google meeting, or every characteristic available from Google Calendar events.
  - **FR-005.2.7: Data Storage.** Storing the given information by the user where he specifies, or if no Google Drive folder is specified the corresponding one depending on the received information characteristics.
- **FR-005.3: Combined actions.** The chatbot shall be able to perform complex requests that involve multiple dynamic action execution of different tools. It shall be able to create the correct order of tool usage and perform a complex chain of tooling to do what the user has requested.

FR-006: User-Centric Adaptability.

The system shall adapt its responses and functionalities based on the user's role and historical interactions to provide a personalized experience. For this, it is essential to keep a rigurous register of the context and history of conversation.

3.2. Non-Functional Requirements

Apart from the functional requirements, the created system has to fulfill a series of characteristics, related with how the system performs rather than what it does. These requirements are essential for the quality, maintainability, and operational success of the digital assistant.

3.2.1. Performance

NFR-001: Response Time for Chatbot Interactions

The system shall respond to user queries via the primary agent chatbot within an average of 30 seconds or less for 95% of interactions.

- **NFR-001.1: Complex Query Latency.** For complex queries requiring synthesis of information from multiple MCPs , the response time shall not exceed 5 minutes.

NFR-002: Daily Planning Generation Time

Automated daily personalized work plans will be created and available in Google Drive by 4:15 AM CEST every day.

- **NFR-002.1: Data Collection Window.** The system will complete all data collection on Trello, GitHub, and Google Calendar for daily planning in under 30 minutes.

NFR-003: Email Processing Throughput

Incoming emails shall be processed (categorized, relevant documents created) within 5 minutes of receipt for 99% of cases.

NFR-004: Meeting Transcript Analysis Time

Meeting transcripts (up to 60 minutes in duration) shall be fully processed (summarized, analyzed, and private data extracted) within 15 minutes of upload.

NFR-005: Concurrent User Support

The system shall support at least 20 concurrent active users interacting with the primary agent chatbot without a degradation in response time or service quality.

3.2.2. Reliability and Availability

NFR-006: System Availability (Uptime)

The Missio OS assistant shall be available and operational at least 99.9% of the time (24/7).

- **NFR-006.1: Scheduled Downtime.** All scheduled or maintenance downtimes shall be communicated at least 24 hours in advance, and scheduled during non-peak hours for Missio IA.

NFR-007: Data Consistency and Integrity

The system shall guarantee the consistency and integrity of all data processed and stored across integrated platforms (for example, Trello, GitHub, Google Drive). Changes to data shall be atomic, and changes to data data would be reliable.

NFR-008: Error Handling and Recovery

The system shall handle any exceptions and errors, and ensure that failures of one part of the system do not propagate to other modules.

- **NFR-008.1: Automated Recovery.** In case of a failure, the system shall automatically attempt to recover or restart within 5 minutes.
- **NFR-008.2: User Notification of Failures.** Users shall be informed of any critical system failures affecting their interactions.

3.2.3. Scalability

NFR-009: User Scalability

The system shall be capable of scaling to support up to 50 active Missio IA team members within the next two years without having to be redesigned.

NFR-010: Data Volume Scalability

The system shall effectively process and manage increasing amounts of data, including :

- Up to 1,000 emails per day.
- Up to 500 Trello tasks actively managed.
- Up to 20 hours of meeting transcripts per week.

3.2.4. Security

NFR-011: Authentication and Authorization

The system shall have strong authentication methods to make sure that only authorized users have access to certain functionalities and data.

- **NFR-011.1: Least Privilege.** The system shall implement the least privilege principle, so all components and integrations are only provided the minimum permissions needed.

NFR-012: Secure Credential Management

API keys, access tokens, and other types of sensitive credentials shall be secured, instead of writing directly in the application, with a method to access it securely (secret management services).

NFR-013: Vulnerability Management

The system shall utilize the security features of cloud providers and follow the secure coding practices to minimize vulnerabilities.

3.2.5. Privacy

NFR-014: GDPR Compliance

The system shall be designed and operated in a justified manner with the GDPR and related to the processing of personal data.

- **NFR-014.1: Data Minimization.** The system shall collect and process personal data only to the extent strictly required to deliver its stated functionalities.
- **NFR-014.2: Purpose Limitation.** Personal data shall be collected, and particularly private relationship data, shall only be used for the stated purpose of facilitating personal interaction. It shall never be used for another purpose without explicit, separate consent.

NFR-015: Secure Handling of Private Data

The specific private relationship data derived from meeting transcripts shall be held in an environment with highly restricted access to designated personnel with a need to know.

- **NFR-015.1: Access Logging.** All access, by any personnel, to the private data shall be logged for auditing.

3.2.6. Maintainability and Extensibility

NFR-016: Modularity and Component Independence

The architecture of the system shall be modular with precise interfaces between components to allow independent development, testing and upgrades.

NFR-017: Code Readability and Documentation

The code shall follow coding standards and best practices, along with inline comments and separate documentation, to facilitate understanding of the code's functionality by future developers.

NFR-018: Ease of New MCP Integration

The architecture shall allow the straightforward addition of new Model Context Protocols as the business needs change or growth.

3.2.7. Usability

NFR-019: Intuitive User Interface

The primary agent chatbot interface shall be intuitive and easy to use for a user to effectively interact with Missio OS.

NFR-020: Clear and Concise Responses

Responses from the chatbot and generated reports shall be clear, simple and easy to understand, while avoiding complex terminology, if possible.

3.2.8. Cost

NFR-021: Operational Cost Efficiency

The monthly costs related to operating Missio OS (cloud computing, API usage, and third-party services) shall be kept within a set budget, while optimizing resources. The costs shall be lower than those of market competitors.

- **NFR-021.1: Resource Optimization.** The system shall work to use resources effectively (like using serverless computing, limited use of APIs) to minimize costs of operation.

3.3. Architecture Overview

This section has the objective to provide a clear and high-level vision of Missio OS system structure, showing its component and how they interact to achieve its functionalities.

3.3.1. High-level system overview

Missio OS is composed of a system of AI agents that are properly orchestrated. The main agent, which users will interact with through the interface is the orchestrator. This main agent supervises everything that occurs in the system, it can see everything the other agents do (such as the planner/blocker detector and the email opportunity detector), and can change any of their instructions. This agent also has access to various Model Context Protocols (MCPs) like all of the other agents. These MCPs are what gives the agent the capacity to perform beyond a basic conversational assistant. These MCPs cover model access to copy/modify company files, task management, interaction with GitHub repositories, calendar interaction and many more. This means the system is more modular and can access more functionality more efficiently, in any or across relevant data sources.

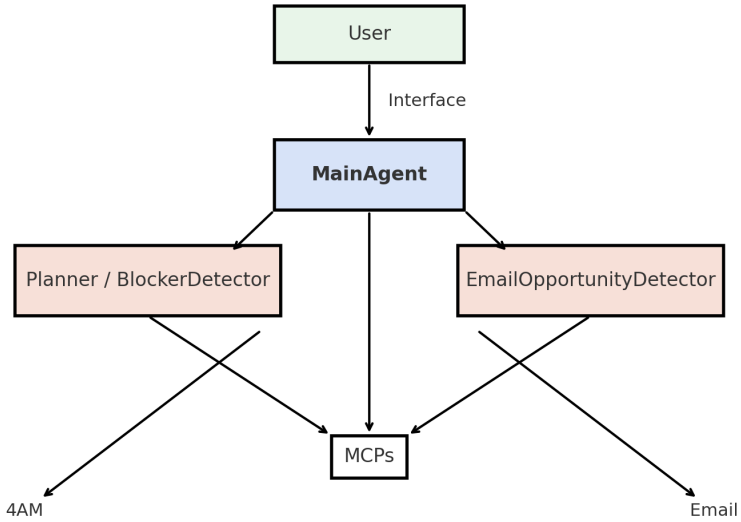

Fig. 3.1. High-Level Architecture of Missio OS.

Figure 3.1 shows how the system is orchestrated by the Main Agent. Users can only communicate with it, via the interface, in a conversational way.

The other two agents, are triggered by events: The Planner/BlockerDetector is activated every day at 4am, and if the day is not Saturday or Sunday, it conducts its corresponding analysis, notifying the Missio IA team by gmail, and saving the conclusions in a Google Drive file. The EmailOpportunityDetector is activated every time a email is received by a company member. It conducts its own analysis, and saves it in a Google Drive file too.

The way the Main Agent communicates with the other two agents is not a conventional, direct way. It does it through Google Drive and the files those agents create. So, every time the user asks something about the actions of those agents, the Main Agent obtains it through the detailed descriptions in the Google Drive generated files. If the user has any feedback or desire to change the way the agents are behaving, he can tell the Main Agent through the chat interface, and change the way they behave by changing their instructions. They both have a Google Drive file from which they take part of their instructions, so their behavior is modifiable. The main agent´s instructions, however, can´t be changed. This is because they are much more delicate for the correct functioning of the system.

All these agents have access to all the tools they need through several MCPs . These are:

- Gmail MCP, that allows to access to all the emails received and sent by the company.
- Trello MCP, which allows access to the task management system used by the company.

- GitHub MCP, that brings access to all the code repositories of the company.
- Google Calendar MCP, where the agents can access to all the programmed events.
- Google Drive MCP, which is the heart of this system. All the companys data and information are stored in Google Drive files, in a very ordered structure. It also allows the communication between the Main Agent and the other ones.
- FireCrawl MCP, which allows internet searches, and more deep research tasks to be executed.

3.3.2. Core Architectural Components

This section describes the main components that make up Missio OS. These components cohere together to run the entire system and handle all the different tasks and communications.

Main Agent

The Main Agent, acts as the brain of Missio OS. It is the focal point for users to communicate with the system, from a chat interface. Its function is to detect and interpret a user's request. Once this is identified, it acts using the corresponding MCPs and tools, to respond to what user has asked. Often, the Main Agent will synthesize input obtained from multiple MCPs, when the user requests information that draws on multiple inputs, and relay a synthesized, cohesive output, or perform an intricate action appropriate to a complex response. This is designed not only to enhance the user experience but also to orchestrate complex workflows seamlessly.

As the MCP connections take some time, the Main Agent API has two endpoints, a POST /connect one, when it takes the email the user is using in the interface (know thanks to the authentication methods implemented), and connects to every MCP. Once it finishes, it returns an id that will be used in every POST /chat request. These requests are the ones that allow to communicate with the agent. When a connection expires, it is automaticall launched a POST/connect one, so that no error is raised and the interface is always operative.

To keep a permanent context, that is stored even the user closes the interface and opens it again in a different day, every time the Main Agent returns a response, it adds the response and the question asked in a json file stored in a cloud bucket. When it loads the context, it does not load all of it if it is too long, it only takes the last 40 interactions. This is done to avoid excessive context problems.

Specialized Agents

These are the EmailOpportunityDetector and the Planner/BlockerDetector agent. They are triggered by specific events and perform very concrete and complex workflows to do their purpose. They can ba altered by the Main Agent, and communicate with it by creating in Google Drive very detailed files explaining what they have done. Likt this, they don't have to be working 24/7 to assist Main Agent requests. They just get activated when it corresponds (4AM or received email), and document what they have done so that the Main Agent can access it.

Model Context Protocols

They are the key part of this project, and what have made it possible. instead of creating a lot of functions with all the API integrations and making the agents return json format answers to call them and creating very complex workflows, they allow the agent to directly use tools, and see their result before writing an answers and stop working. They are the Gmail, Google Drive, Google Calendar, GitHub, Trello and FireCrawl MCPs that have been explained before.

Artificial Intelligence Core

The smart features in Missio OS come from its Artificial Intelligence Core. This core depends on Large Language Models, or LLMs. An LLM can read what a user writes, understand the intent, and then build a reply that sounds natural. Because of this, the system feels less like a script and more like a real assistant. In short, the LLM is the part that lets Missio OS listen and speak in everyday language.

For the project the o4-mini model from OpenAI was selected. It was chosen after comparing performance, price, and speed. The model follows a clear chain of thought, which helps it break a problem into steps before giving an answer. That careful reasoning is useful for hard questions or when the Main Agent must create a long summary. At the same time, the model runs fast enough so users do not wait and cheap enough so we stay within budget.

Chatbot interface

Another very important component is the interface that allows users to use the product. It has a user-friendly design and is very simple to use. It includes a login screen, with authentication via email-password or Google. Once authenticated, the user enters the chat screen, where it can communicate with the Main Agent. The frontend autmatically handles connections with the Main Agent API, making new ones when the current ones expire. This enhances efficiency and reduces latency.

3.3.3. Key System Workflows

Missio OS tackles demanding situations through a set of clearly-defined steps, yet the whole process feels quite fluid because each module knows exactly when to step in. To illustrate that coordination I will describe two typical workflows that I have used during my internship.

The first one begins the moment a meeting ends. A Google Meet recording is transcribed and the text goes straight into the system. Missio OS checks who attended and decides whether the external party is already a client or a brand-new lead. It does this by comparing the names with the folders we keep in Google Drive. If the match is not found, the platform creates a new folder, labels it as a potential client and stores the transcript there. Once the file is safe, the system writes a meeting summary, points out strong and weak moments, and suggests concrete improvements. If a proposal must be written, it prepares 5 mock ups so that a team member can later chose the best one. If action items pop up it opens the right Trello board, writes the tasks, assigns teammates, and even spins up a fresh GitHub repository when new code is required. For each coding task it drops in a first draft with plenty of comments, so whoever picks it up starts from a tidy baseline instead of a blank page.

The second important workflow happens when we are ready to launch a new product that we've developed in Missio IA. When I tell the system to find potential clients for a certain product, it kicks off a specialized process. Often, we build products because we think they are good ideas, and once they are polished, it's time to sell them.

So for this, the system first looks for the product's descriptions in Github, as well as related Google Drive documents. The system must understand what the product does, and what benefits the product will provide. The system then uses its superior research capabilities to perform an in-depth search for potential companies that are likely interested in our product. The system identifies these companies, and gathers their relevant contact information. If I give the system the directive to contact the companies, the system takes over professional crafting and delivering emails to the companies, maintaining a serious tone and the professionalism which is standard for communications from our company. This workflow shows an automated piece of our sales process, from lead identification to first contact.

3.4. Technology Stack

The software that powers Missio OS is built on a set of technologies chosen for their maturity, cost, and how well they fit together. All the back-end agents are written in Python, a language that is easy to read yet strong enough for complex data work. In the agents, OpenAI client v1.79.0 is used to send prompts and receive answers, while the openai-agents-mcp v0.0.8 package is used because it gives the extra layer that makes

Multi-Call Programs work, and allows to incorporate MCPs and their functinoalities to agents. Thanks to that wrapper an agent can ask for several tools in a single request, keep a short memory of earlier steps, and then return a unified reply. Inside the same service, the endpoints are exposed with FastAPI and Uvicorn lets them be served asynchronously, which means many users can work at the same time without blocking each other. Most calls to the LLM are still synchronous because the model needs the full prompt before it can think, but the web layer stays reactive.

On the front-end the stack is based on Next.js combined with TypeScript. Next.js brings a neat, modular structure where pages, shared services and middleware remain well organised, so later tests do not turn into a maze. TypeScript adds static typing, which catches many small mistakes before they reach production and becomes even more valuable when the project grows.

The only back-end endpoint exposed to the client is the Main Agent API. This service runs inside a Google Cloud Run container. Cloud Run behaves almost like a fully serverless platform: an image is pushed, a few environment variables are set, and the platform handles scaling up or down. As a result, no charges appear while the service stays idle, yet extra replicas spin up automatically during peak traffic.

Each auxiliary agent is deployed where its profile fits best. The Planner/Blocker Detection agent is packaged as an HTTP-triggered Firebase Function and launched every day at 04:00 by Cloud Scheduler. This setup avoids keeping a full server online just for a daily routine and lets Google manage the timing logic. The EmailOpportunityDetector agent is deployed as a Pub/Sub-triggered Firebase Function, to ensure that it is only activated when an email is received by any member of the team, which helps reducing costs too.

Model Context Protocols are hosted in Google Cloud Run Containers because they sometimes have to always be available to use when an agent needs them, but they don't need to be active all day, when there is no traffic. This helps maintain costs within budget, and also, Cloud Run is a very reliable service.

Persistent context lives in a Google Cloud Storage bucket. One JSON file per user e-mail keeps personal data isolated, so the Main Agent loads only what it needs. Isolation prevents cross-client leaks and makes backups straightforward; lifecycle rules even move older versions to cheaper Nearline storage. Static assets for the web application follow a different path and are served through Firebase Hosting, which offers HTTPS and a worldwide CDN out of the box. User authentication also remains under the Firebase umbrella, so sign-in and token refresh rely on a battle-tested service instead of bespoke code.

This technology mix offers several advantages. All key components are managed services, which cuts maintenance overhead and frees the team to focus on features, something very important in start-ups that are low on personnel and have a lot of things to do. Automatic scaling is built-in: Cloud Run adds replicas on demand, while Firebase Hosting and Functions follow the same pay-as-you-go model. Finally, the toolchain, Python, TypeScript, container images, and familiar Google Cloud products, is common in the

industry, so new contributors can get productive quickly. Altogether, the stack keeps Missio OS reliable, cost-predictable, and ready for future requirements.

3.5. Security and Privacy

Security and privacy are fundamental tenets of Missio OS design and daily operation. The platform processes private client conversations, internal confidential plans, and personal relationship analysis, and it is imperative to safeguard that information for trust and business continuity. Every measure of protection is chosen with the same aim in mind: to keep data out of unauthorised hands, to prevent accidental loss, and to prevent any abuse that could cause detriment to either clients or the organisation.

Data travels constantly, moving between components inside Missio OS and toward external services. During that journey every packet is wrapped in Transport Layer Security, which shields it from interception or tampering. Once the information comes to rest, it is still protected. Files stored in Google Drive rely on Google's native encryption, while auxiliary databases activate encryption at the storage layer as well.

Keeping the doors locked starts with strong authentication. Users sign in with their existing Google Workspace accounts through OAuth 2.0, which means Missio OS never sees raw credentials. Only emails with a "@missioia.com" domain can access the interface, so this is an extra layer too that will allow only members of the organization to use it. Within the platform, permissions follow the least privilege principle. Each user, service account, or internal module is given only those privileges required for its activities, not additional ones. Role based access control differentiates between administrators and regular operators, and all internal calls between the orchestrator and the respective agents are authenticated with service accounts or safely stored keys.

Privacy principles guide all design decisions, including the requirements of the General Data Protection Regulation. Missio OS promotes data minimisation, collecting only the personal data which is absolutely necessary for its intended purpose. The same principle of purpose limitation prevents reuse of that data without the explicit consent of the individual. Mechanisms are also in place to support the rights of data subjects, so any user can request access to their records, or ask for those records to be erased when the law allows.

So, Missio OS is designed with a strong commitment to privacy, adhering to key data protection regulations such as the GDPR [57] which is especially relevant for operations within the European Union.

Also, the code has been done following securite coding guidelines, such as not hardcoding any key, and using secrets and venv files.

A secure system is never finished, therefore Missio OS follows an ongoing routine of hardening and vulnerability management. Secure coding guidelines are applied, dependencies are updated promptly, and rigorous input validation blocks common injection attacks. Also,

all the agents have been instructed with some malicious actions that they can not trigger under any circumstance, such as erase a folder in google drive or a GitHub repository, among others.

The cloud platform itself brings added protection through firewalls, fine-grained identity management, and detailed security logs. Continuous monitoring captures any unusual activity, allowing the team to respond quickly before a small warning turns into a real incident.

4. IMPLEMENTATION

This chapter explores the technical details of the Missio OS implementation, outlining low-level design decisions and the specific technologies used to build each component of the system.

4.1. Connector Modules (MCPs)

Before the recent iterations of specialized Model Context Protocols took hold and were widely assumed, our initial attempts to construct Missio OS relied upon a traditional chatbot country/protocol architecture. Specifically, we developed individual, manual API integrations for every single external service and connected each interaction to multiple combinations of functions in order to support specific conversation(s) and accomplish specific tasks in each context in each external service. While the integrations worked, the aforementioned process resulted in an awkward, inefficient, and cumbersome process for assembling (connecting) all of the manual integrations together. Creating and maintaining a myriad of manual integrations was cumbersome and challenging to maintain—let alone scale. However, when the definition and value of the MCP concept emerged, it was crystal clear how to take advantage of it to integrate multiple external services and improve the overall efficiency of the Missio OS system. Therefore, we pivoted and eliminated the hand-assembled integration process and adopted the MCP framework and consistently achieved more efficient and better results.

So, Model Context Protocol servers act in Missio OS as connector modules: they hide the complexity of third-party APIs and expose a single, tool-based interface to the Main Agent. MCP specifies two wire formats, stdio and Server-Sent Events [2]. The latter keeps a long-lived HTTP stream, so the language model can start reasoning as soon as the first bytes arrive; for that reason every connector has been configured for SSE, even when the upstream project originally shipped with stdio.

Each connector targets exactly one external service and advertises a set of domain-specific tools. The Main Agent discovers those tools at run-time, sends requests as plain JSON, and receives structured replies, so its workflow does not change when a service is swapped or upgraded.

4.1.1. Gmail MCP

The Gmail MCP manages every call to the Gmail REST API. It started from the open-source repository in [58], was converted to SSE, and the OAuth flow was replaced by service-account impersonation so that a single connector can reach any mailbox under the @missioia.com domain. To be able to access the correct gmail account and avoid mistakes,

its connection process was modified to receive a gmail account, that it will use during all the requests done during the connection.

4.1.2. Google Drive MCP

Based on the project in [59], the Drive connector originally offered only file listing and basic uploads and modifications, and Google Sheets operations. It now supports SSE and full Google Docs operations (create, write, and update). These extensions let specialised agents store meeting transcripts under the correct client folder and later allow the Main Agent to retrieve them without extra code.

4.1.3. Trello MCP

The Trello connector derives from [60]. Besides the obligatory SSE upgrade, a new assign-member tool maps company e-mail addresses to Trello IDs and attaches them to cards. Standard operations for reading boards, moving cards, and changing statuses come straight from the original implementation.

4.1.4. GitHub MCP

Missio OS uses the official GitHub MCP server [61], but the authentication layer was rewritten to use an organisation token instead of a personal one. With that change the connector can create repositories inside the Missio-Ia organisation, open branches, push code, and raise pull requests with no human tokens in the loop.

4.1.5. Google Calendar MCP

No suitable Calendar server was found, so a new one was implemented. It authenticates through a service account that can impersonate every employee calendar and exposes tools such as list-events, create-event, and update-event. Live updates reach the agent through SSE, which is helpful when daily plans are regenerated on the fly.

4.1.6. FireCrawl MCP

Web-scraping capabilities come from the repository in [62]. The only change required was to change the configuration t enable SSE. The connector gives the agent tools for fetching raw HTML, extracting main text, and following links, which are later summarised or cited in answers.

All external MCP repositories selected for Missio OS already exhibited a mature design, clear documentation, and active maintenance. The selection process compared several

public alternatives and kept only those whose structure and licence aligned with the functional scope of this thesis.

4.2. Agent Orchestration

Orchestration is the coordinated effort that lets several autonomous agents join forces to complete workflows that no single module could handle on its own.

Missio OS places that responsibility on the Main Agent, which acts as a central orchestrator and supervises every other component in the ecosystem. Users interact only with this agent, yet it can observe and adjust the behaviour of the Planner/BlockerDetector and the EmailOpportunityDetector whenever needed.

4.2.1. Design Evolution and Iteration

Before specialised MCP connectors became available, the first prototype relied on hand-written API calls for each external service. Every new feature added fresh code paths, tight coupling, and duplicated authentication logic, which soon proved hard to scale.

Maintaining that approach meant juggling several API versions, different OAuth flows, and fragile error handling. Extending the system to an extra platform required yet another bespoke client, so a more modular layer was needed.

Model Context Protocol servers replaced the ad-hoc glue with independent, domain-specific modules. Each MCP hides low-level details and offers tool endpoints that the Main Agent can call directly. With the connectors in place, the orchestrator now focuses on intent recognition and task delegation instead of wrestling with raw REST calls, which makes the whole architecture easier to maintain and extend. Also, some new models with chain of thought, such as the o4-mini that is used, have been trained for correct tool calling, something that combines very good with the capabilities that MCP servers add to an agent.

4.2.2. Prompting Strategies for Agent Behaviour

LLMs are “programmed” through carefully crafted prompts rather than imperative code. The Main Agent prompt spans roughly two hundred lines and mixes system directives, few-shot examples, and dynamic user instructions. It explains when to invoke each MCP tool, how to chain calls, and even how to react when a Google Meet transcript lands in Drive: the agent must file the document, summarise it, extract action points, and open Trello or GitHub tasks whenever necessary. Few-shot pairs illustrate typical user requests so the model recognises intent quickly, while concise system rules keep answers consistent across sessions. Because the context is stored in a bucket after every interaction, the prompt only reloads the last forty turns, avoiding excessive context length.

4.2.3. Planner/BlockerDetector

Every weekday at four in the morning a Cloud Scheduler trigger wakes the Planner/BlockerDetector. The agent queries Trello, GitHub, and Google Calendar via their MCPs to gather the current state of tasks, code progress, and team availability. With that snapshot, and with knowledge of each member's strengths encoded in an auxiliary table, the agent drafts a personalised work plan, flags stalled cards, and proposes fixes for any blockers. Findings are written to a Drive document and a short summary reaches the team by e-mail. This indirect channel means the agent can stay dormant the rest of the day, saving compute while still keeping the Main Agent informed.

4.2.4. EmailOpportunityDetector

A Pub/Sub subscription forwards every incoming message to the EmailOpportunityDetector. When the sender belongs to a company domain, the agent checks Drive to see whether it is an existing client or a prospect; if the name is new a folder and an opportunity document are created automatically. For regular enquiries the agent drafts a reply and stores it as a Gmail draft so a human can review before sending. Complex leads generate a proposal template in Drive that company staff can refine, again leaving a trace the Main Agent can consult later.

4.2.5. File-Mediated Communication

Agents do not talk to each other directly. Instead they publish detailed reports and instructions as Google Docs inside Drive; the Main Agent reads those files when answering follow-up questions or when it needs to adjust secondary agents. This document bus avoids race conditions, supports offline audits, and lets any module refresh its logic by editing a single Drive file rather than redeploying code. Because every report follows a fixed template, the Main Agent can parse information reliably and merge insights from different sources into a cohesive response for the user.

The orchestrated system delivers an updated daily plan, detects blockers before they escalate, and answers client e-mails with minimal human effort. All results are stored in Drive for transparency, while notifications keep the team informed in real time.

4.3. Backend Services

The backend of Missio OS is designed to give the orchestration layer a strong and reliable foundation while keeping day-to-day operations simple for developers. At its heart sits a FastAPI application that exposes two main endpoints: POST/connect, which authenticates a user and returns a session identifier, and POST/chat, which receives messages linked to that identifier and streams the reply. FastAPI was chosen because it combines high

throughput with an intuitive, OpenAPI-driven design and the strict data validation that Pydantic offers. All routes run asynchronously, so the server can handle many conversations in parallel, and Cross-Origin Resource Sharing is explicitly enabled for localhost during development as well as for the two production hosts served from Firebase.

CODE 4.1. Core FastAPI endpoints with CORS configuration.

```python
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# ----- Pydantic models -----
class ConnectReq(BaseModel):
    email: str

class ConnectRes(BaseModel):
    session_id: str

class ChatReq(BaseModel):
    session_id: str
    message: str

class ChatRes(BaseModel):
    session_id: str
    reply: str

# ----- FastAPI app with CORS -----
app = FastAPI(title="Missio OS Chat API")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://missio-os.web.app", "http://localhost
        :3000"],
    allow_credentials=True,
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["*"]
)

# ----- Endpoints -----
@app.post("/connect", response_model=ConnectRes)
async def connect(req: ConnectReq):
    # CONEXION LOGIN...
    return {"session_id": session_id}

@app.post("/chat", response_model=ChatRes)
async def chat(req: ChatReq):
    # CHATTING LOGIC...
    return {"session_id": req.session_id, "reply": reply}
```

The following sequence diagrams show the life cycle of the two endpoints exposed by the Main Agent backend. Figure 4.1 covers the connection workflow and Firgure 4.2 covers

the chat interaction.

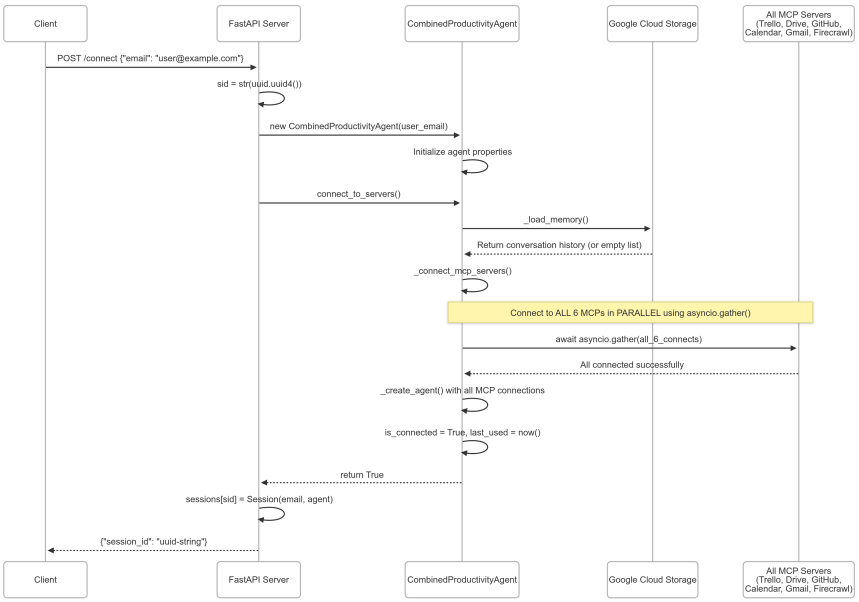

Fig. 4.1. Sequence Diagram of POST/connect Endpoint.

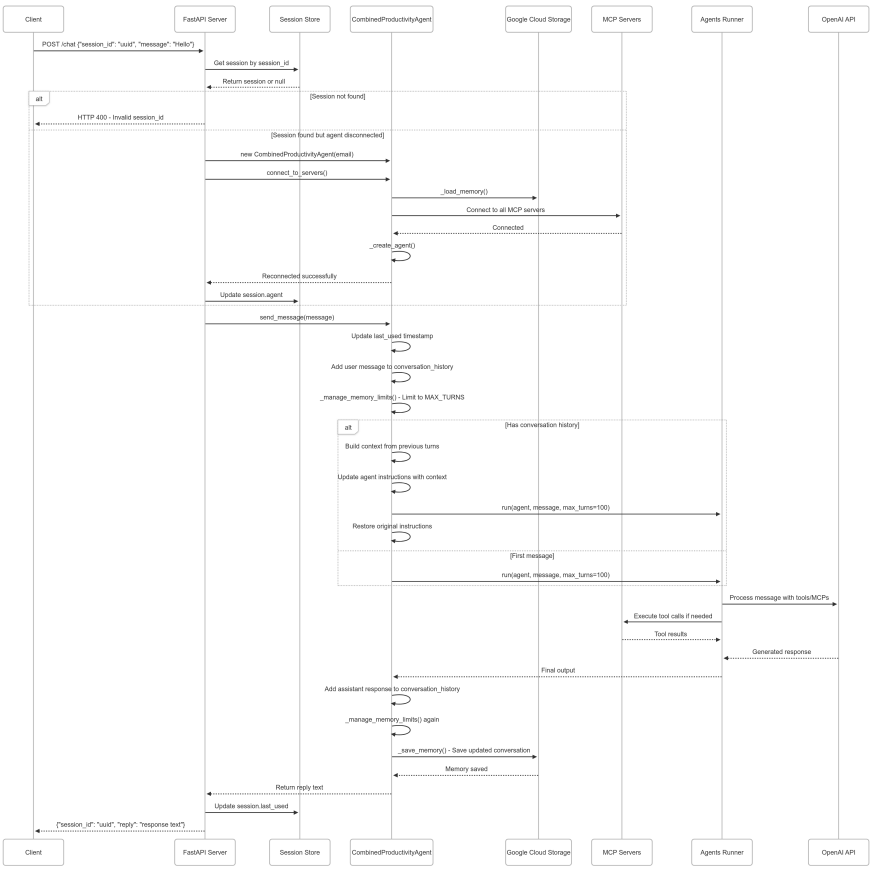

Fig. 4.2. Sequence Diagram of POST/chat Endpoint .

Every backend component lives inside its own Docker image. This simple rule guarantees

that what runs in a laptop behaves the same in staging and once deployed. Images are pushed to Google Container Registry and then launched on Cloud Run, which feels almost serverless because Google automatically scales instances to zero when traffic stops and back up again within seconds when new requests arrive. The pay-per-use model matches the irregular workload of an agent platform, which can stay quiet at night yet spike during a product demo.

Two secondary agents follow a different path. The Planner / BlockerDetector, responsible for the four-o'clock daily review, and the EmailOpportunityDetector, triggered by Pub / Sub whenever a new message reaches the shared inbox, are written as Firebase Functions v2. Packaging them that way removes the need to maintain separate servers for tasks that run on a strict schedule or react to a single event, while still letting the code share the same Python dependencies and OpenAI client as the central service.

Persistent data is split by nature. Structured records, such as configuration flags, user preferences, and the private relationship details summarised from meetings, are stored in Google Cloud Firestore. The document model fits well because each piece of information evolves independently and must sometimes be delivered to the front end in real time. All files that carry heavy content such as meeting transcripts, generated summaries, or daily plans stay in Google Drive, which already provides fine-grained permissions and integrates smoothly with the Drive MCP. This division keeps queries fast for small records while large documents benefit from Drive's built-in versioning and collaborative features.

By combining a modern asynchronous framework, containerised deployments, serverless execution, and cloud-native storage, the backend stays lean, scales with demand, and lets the development team focus on improving agent logic instead of managing infrastructure.

4.4. Front-End and Authentication

The user experience of Missio OS is delivered through a web application built with Next.js. The choice of this React framework comes from the balanced combination of server-side rendering, static generation, and a component model that keeps the code-base modular. While search-engine optimisation is not important for an internal tool, server-side rendering still improves first-paint time, which creates a perception of responsiveness and agility even in a slower connection. Each visual object lives in its own TypeScript component, which keeps layouts predictable and safe from refactoring with static typing.

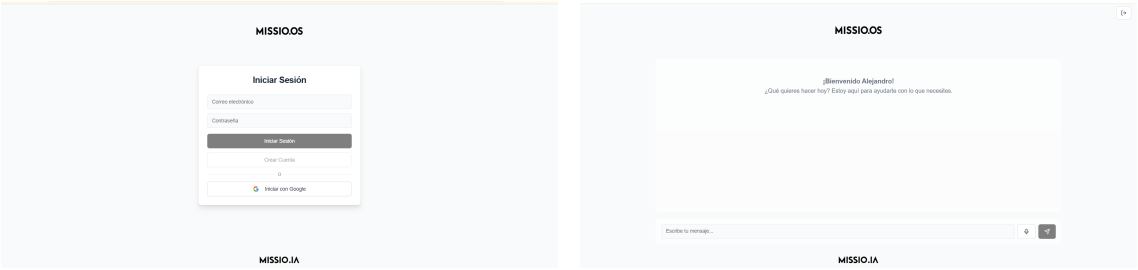

Fig. 4.3. Login page (left) and chat page after successful authentication (right).

Figure 4.3 shows the two screens that users meet most often. The first is a clean login form that supports e-mail/password and Google Sign-In. Once authentication succeeds a personalised greeting appears (written in Spanish, the main language at Missio IA) and the central chat area opens. Below the text box a small microphone icon lets the user dictate messages; Speech-to-Text conversion using OpenAI Whisper API converts the audio and sends the resulting string to the Main Agent, which is handy when typing is not convenient.

User identity is managed by Firebase Authentication. All sign-in flows, token refresh, and session storage are handled by the Firebase SDK, so the front-end contains almost no custom security code. Integration with Google Workspace means employees can enter with a single click. After login Firebase delivers a JSON Web Token that the browser attaches to each request sent to the back-end, where FastAPI verifies the signature before processing.

Together, Next.js and Firebase Authentication provide a fast, safe, and low-maintenance front end. The code stays approachable for new contributors, performance remains stable as traffic grows, and the development team can focus on adding agent features instead of reinventing login flows or client-side routing.

4.5. DevOps and Monitoring

Reliable DevOps routines and solid monitoring keep the entire Missio OS stack healthy day after day. Because most components already live inside Google Cloud, the platform's native services became a natural choice and, in fact, played a big part in the decision to host the project there. Utilizing a single provider for build, deployment, logs, and metrics means a lower number of moving pieces and faster troubleshooting time when something inevitably breaks.

Similarly, the Google Cloud Build powers the continuous-delivery pipeline. Each push to the main GitHub repository fires a build trigger which retrieves the source, executes Python or Node dependency installations, and builds a new container image. Unit tests and a small integration suite run inside the same build step so a failing commit never reaches production. If everything turns green the image is tagged and deployed automatically to Cloud Run. This flow removes manual steps, guarantees that each revision is reproducible, and cuts the time between writing code and seeing the result online.

Once containers are live, the exhaustive logging will be the first line of defence against silent failures. All stdout and stderr streams from Cloud Run are sent to Google Cloud Logging along with structured entries emitted by Firebase Functions and the individual MCP servers. A single dashboard can therefore search across the whole platform, filter by request ID, or inspect stack traces. Retention policies keep thirty days of verbose logs for rapid debugging while archiving older entries at a cheaper tier.

Metrics are collected by Google Cloud Monitoring. Standard indicators such as latency, error rate, and container CPU load are enabled out of the box, custom metrics record the number of LLM calls, token usage, and the size of the context bucket. Alerting rules fire when thresholds are crossed, for example a sustained > 5% error rate or a sudden spike in memory consumption, and notify the team through the Missio IA Slack workspace. This early warning system helps catch problems long before users notice them.

Together, automated builds, centralised logs, and real-time alerts create a feedback loop where issues surface quickly and fixes roll out with minimal friction. The result is a service that remains responsive, cost-efficient, and trustworthy even as new features arrive and traffic patterns change.

4.6. User Feedback and Iterative Improvements

The development of Missio OS follows a cycle where real user feedback shapes every new release. Early prototypes were rolled out to a small group of Missio IA teammates who tried them with a few requests they could think about, and reported the performance. Once a more developed version was ready and deployed, about a few weeks ago, they started using it and recording all the problems, ideas of improvement and misbehaviors found. Once a week, they reported them in a meeting, and a new version of MISSIO OS was aimed to fix them.

For example, the initial prototype did not have the recording button with the speech-to-text in the user interface, and the idea to add it was given as one of this feedback. It was a perfect example of how this cycle helps develop a better product, because it made the interface easier to use if the user wants to send a very long message as request.

Indeed, most of the feedback is behavior related and can be fixed by adjusting the prompt. There are very complex requests that require calling multiple tools in the correct order and concatenating results, so when trying a new kind of compliocated request misbehaviors might happen.

Although the product is now stable and feature-rich, this improvement cycle is not meant to stop. Feedback collection and weekly triage will remain part of normal operations, and any teammate who proposes a useful tool, a fresh automation, or even a brand-new agent will see that idea reviewed against the roadmap. When the benefits are apparent and the cost is acceptable, it will be able to join the next sprint and travel through the same continuous integration and continuous deployment (CI/CD) pipeline as earlier changes.

This allows Missio OS to adapt to additional use cases, without losing sight of the design principles that deal with consistent past and present practice in the moment.

5. EVALUATION

Chapter 5gives an overview of an evaluation of Missio OS for the purpose of empirically investigating the performance, efficiency and effectiveness of Missio OS in automating critical operational work. The evaluation aims to objectively quantify Missio OS capabilities, particularly when leveraging different Large Language Models, to inform important architectural decisions and to confirm that the project goals were satisfied. All the models under the evaluation are entered the same prompt, that has been refined to make the agent success the greatest number of times.

5.1. Evaluation Methodology

The entire evaluation process was designed to test the appropriateness of using different Large Language Model (LLM) candidates for the primary AI backend of Missio OS, with the ultimate goal of identifying the best suited LLM candidate for production use. The evaluation was performed by simultaneously evaluating Missio OS (with various LLM candidates) using a consistent set of twenty tasks that provided a variety of operational scenarios and complexities that relate to the daily workflows of Missio IA. The tasks allowed for a more thorough and objective way to measure and compare based on the primary performance indicators. Each model will be asked to perform each task one hundred times and results will automatically evaluated by an AI, to be preserved and supervised by a human at a later time.

Simple tasks (10):

1. Summarise the latest unread e-mail in the inbox (Gmail MCP).
2. Draft a short professional reply to that e-mail acknowledging receipt (Gmail MCP).
3. Report the current status of the “Dorna Sports” card on the Sergio-Tabo board (Trello MCP).
4. Create a new Trello card in the “To do” column of the Sergio-Tabo board titled “Review Data” (Trello MCP).
5. Locate the document named “Alejandro Taboada” inside Blocker-Hunter > Personas on Drive (Drive MCP).
6. Create an empty Google Doc titled “New Project Proposal Outline” inside the Blocker-Hunter folder in Google Drive (Drive MCP).
7. Tell me my next meeting today (Calendar MCP).

8. Check the status of the `main` branch for the Missio-OS repository (GitHub MCP).
9. Scrape the headline and first paragraph from `https://www.acolad.com/en/services/translation/impact-ai-globalization.html` (Firecrawl MCP).
10. Define "MCP server" in simple terms (direct LLM, no MCP).

Medium tasks (5):

11. Summarise yesterday's e-mail from product@mail.bubble.io and save the summary as "Client Email Summary – [Date]" in the Blocker-Hunter folder (Gmail + Drive MCPs).
12. For my next calendar event, create a Trello card on the Sergio-Tabo board with a due date one day after the meeting (Calendar + Trello MCPs).
13. Read the Google Doc called "response" and produce a three-sentence executive summary (Drive MCP + LLM).
14. E-mail sergio.rodriguez@missioia.com to say the "Dorna Sports" card has moved to Done (Trello + Gmail MCPs).
15. Visit the Wikipedia page for "Artificial Intelligence" and extract the first-paragraph definition (Firecrawl MCP + LLM).

Complex tasks (5).:

16. Process a full meeting transcription: store the file, summarise it, extract action items, and create Trello tasks as needed (Drive + LLM + Trello).
17. A new product sits in the Missio-OS GitHub repo. Research three to five companies that might adopt it and list their main contact e-mails (GitHub + Firecrawl + LLM).
18. Generate my optimal daily plan and detect any blockers across company tasks (Trello + GitHub + Calendar + LLM).
19. Handle the latest customer-complaint e-mail: draft a polite response and open a high-priority Trello card summarising the issue and reply (Gmail + LLM + Trello).
20. Produce a comprehensive status report for the Missio-OS project, merging recent activity from Trello boards and GitHub repositories and flagging upcoming roadblocks (Trello + GitHub + LLM).

As it can be seen, all this tasks affect only Trello tasks in one specific board, and Google Drive folder, and repositories related to Missio-OS. This was done so that in case of a misbehavior of any of the agents happens, it can easily be located and fixed.

5.2. Models Compared

All experiments focus on models supplied by OpenAI. The company has a preference for this LLM supplier and it is the most oftenly used one. Also, keeping the choice inside one provider simplifies billing, reduces the risk of sudden behavioural differences, and lets the same SDK cover every candidate, so even if it limitates the models to use, it facilitates the evaluation process. Three models were finally placed under test because each offered a distinct mix of price, speed, and reasoning depth that fitted one of Missio IA's common workloads

The first model is GPT-4o, announced in May 2024 and positioned as a general-purpose flagship that can answer text, vision, and audio prompts in a single pass. In practice it delivers short response times, under three seconds for typical chat calls, and a middle-of-the-road cost per token, so it provides an attractive baseline for everyday use where scale matters as much as raw accuracy.

The second candidate is GPT-4.1, released to the API in May 2025. This generation refines the internal optimisations of GPT-4o and introduces a mini variant that is twenty-six per cent cheaper for median queries while cutting latency by nearly half. OpenAI's own benchmarks show higher scores on coding, instruction following, and multilingual reasoning when compared with GPT-4o, making the model interesting for workloads that demand fast turn-around without sacrificing depth.

Finally, o4-mini entered evaluation as a lightweight "thinking" model launched on 16 April 2025. It extends the chain-of-thought approach of earlier o-series models by running a longer internal reasoning path and, when images are involved, manipulating them through crops and zooms before committing to an answer. Although o4-mini is smaller than the two GPT-4 variants, early tests confirm that it solves complex planning tasks at a fraction of their cost, so it offers a compelling option when careful reasoning is required but budget is tight.

These three models therefore cover the spectrum from balanced performance, through cutting-edge efficiency, to economical deep reasoning, allowing the evaluation to measure how much value each trade-off brings to the workflows typical of Missio OS.

5.3. Metrics

Three quantitative indicators were tracked for every experiment run.

The first metric is latency. It measures the wall-clock time, in seconds, from the moment the request leaves the front end until the final token of the answer is received. Timing is captured with a high-resolution stopwatch inside the test harness, so any network overhead or queuing delay on the model side is included.

The second metric is task success. Each task has a checklist that defines what a correct

outcome looks like. If the agent fulfils every item (fetches the right file, creates the card in the right column, writes a summary with the requested length) the run is marked as successful; otherwise it is counted as a failure. After ten repetitions the number of successes is divided by ten and reported as a percentage.

The third metric is cost. OpenAI bills proportionally to the number of input and output tokens, so the test harness logs both counts and multiplies them by the public price per thousand tokens for the model under inspection. This produces a dollar figure for each run that can be averaged without waiting for the monthly invoice to close.

Moshkovich et al. (2025) point out that a solid evaluation of LLM-based agents should always track latency, cost, and a clear measure of task success [63]. Their analytics layer records response time in seconds, keeps a running total of dollar spend by counting tokens, and labels each trace as a hit or a miss. With that trio they compare frameworks without losing sight of either the user experience or the budget.

All twenty tasks are executed a hundred times per model, producing three hundred observations per metric. A simple arithmetic mean yields a representative latency, success rate, and cost for each task–model pair. Finally, overall means are computed across the whole task set so the three models can be compared on a single line of the results table.

5.4. Results

The results of the experiment pooled by the evaluation methodology given in Section 5.1 are presented in this section. The data collected during the execution of a 20-task suite of a single evaluation over the compared LLMs is summarized as sets of tables and figures to provide an illustrative and objective assessment of Missio OS's performance metrics. All these materials will be deeply analysed in Section 5.5.

Model	Success Rate (%)	Latency (s)	Average Cost (USD)
GPT-4.1	79.80	9.54	$1.49
GPT-4o	91.45	10.76	$1.89
o4-mini	89.65	18.24	$0.83

TABLE 5.1. FINAL EVALUATION RESULTS

Table 5.1 shows that the most successful model was GPT-4o, the cheapest one was o4-mini and the fastest one was GPT-4.1

In the following graphs it can be observed how each model performed in terms of success (%) in every type of task (simple, medium and complex).

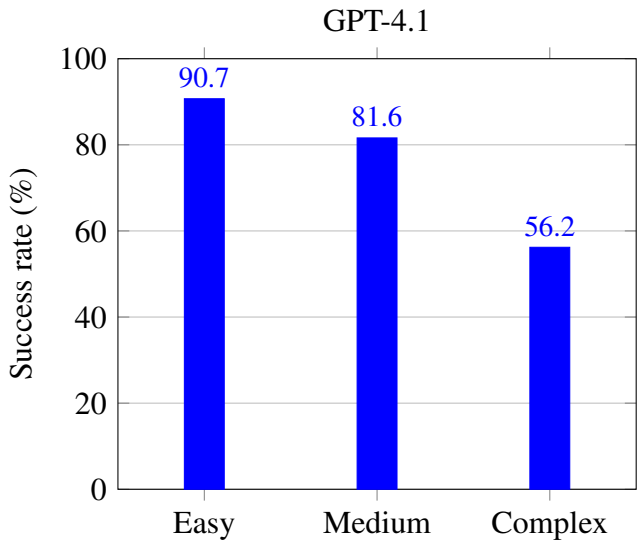

Fig. 5.1. GPT-4.1 success rate by task difficulty.

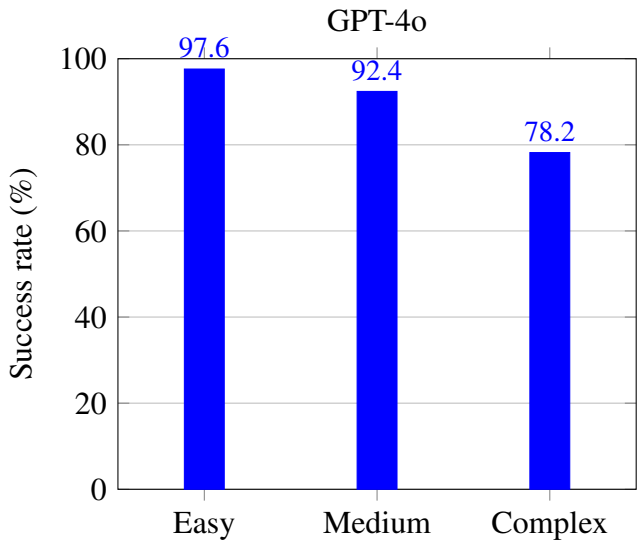

Fig. 5.2. GPT-4o success rate by task difficulty.

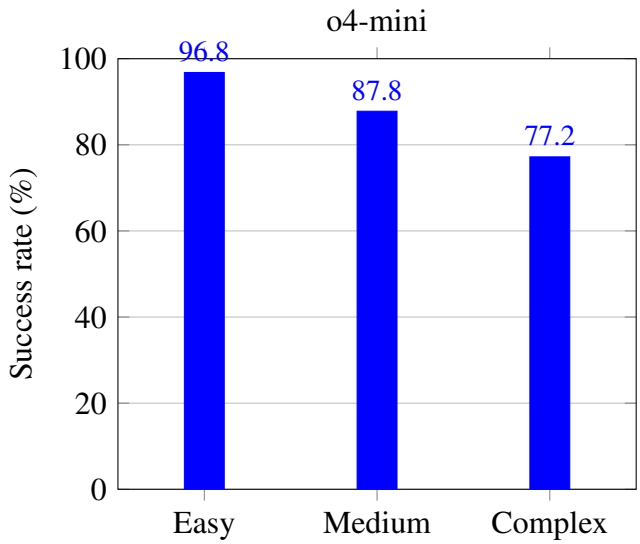

Fig. 5.3. o4-mini success rate by task difficulty.

5.5. Discussion of Findings

This section digs into the numbers presented in Section 5.4 and explains what they mean for Missio OS. The goal is to decide which model offers the best mix of task success, speed, and cost for real-world deployment inside the company.

One of the main conclusions that can be extracted from Table5.1 is that GPT-4.1 falls well below the 85 % t threshold we defined as the minimum for complex automation. Its overall score of 79.8 per cent makes it unsuitable for production, so the model is discarded at once. The main reason is that GPT-4.1 was not trained with the explicit "tool-use" objective that o4-mini and GPT-4o received; without that skill the model struggles to pick the right MCP tool and fails silently in multi-step tasks. By contrast, GPT-4o and o4-mini both stay above 89 per cent across the full task set, and their curves remain high even on complex scenarios. A slight dip for o4-mini on the hardest tasks is visible, as it can be seen when comparing Figure5.2 and Figure 5.3,probably caused by its tendency to write long chains of thought and occasionally wander away from the direct solution, but the difference is small enough to tolerate. What is clear is that both of them are miles ahead of GPT-4.1 in terms of sucess % in any kind of task difficulty, as it can be seen in Figure 5.1, where every bar is significantly lower than in the other two graphs.

Table 5.1 plots the mean cost per task, calculated from input and output tokens and the public prices in OpenAI's pricing sheet [64]. GPT-4o is almost twice as expensive as o4-mini for the same workload, and GPT-4.1 sits in between. Cost matters because many Missio OS operations run unattended every night; a few cents saved per call scale to noticeable monthly savings. On this axis o4-mini is the clear winner.

Table 5.1 also compares the average response time. All three models remain under the 30-second ceiling we set in Section 3, which was very generous, but was the one agreed

in a company meeting, but GPT-4o is the fastest at about 10.8 seconds, followed closely by GPT-4.1, while o4-mini reaches 18 seconds on average. The difference is perceptible but still acceptable: Missio IA staff understand that some workflows involve several MCP calls and prefer accuracy over raw speed. Crucially, o4-mini never crosses the 30-second mark, so it satisfies the NFR-001 (the first and most important non functinal requirement).

Putting the three curves side by side makes the trade-off straightforward. GPT-4.1 fails on accuracy and is ruled out despite its moderate cost and latency. GPT-4o offers the highest success rate and the lowest latency but at a price almost double that of o4-mini. Because Missio IA is a start-up, cost efficiency weighs heavily, and even a small delta multiplied by thousands of tasks per month eats into the cloud budget. o4-mini, while a bit slower, delivers nearly the same accuracy as GPT-4o and does so for roughly half the money.

For the reasons above, the team selects o4-mini as the production model for Missio OS. The assistant's typical workloads are complex enough to justify o4-mini's extended reasoning, and the 18-second average delay is comfortably below the 30-second usability limit. Additionally, model prices tend to drop when newer versions appear; if o4-mini follows that trend its cost advantage will only grow. Should a future release of GPT-4o (or an entirely new series) reach a similar price range, the evaluation pipeline built for this thesis can be rerun to confirm whether a switch makes sense. For now, o4-mini strikes the best balance between precision, speed, and budget, and therefore becomes the default engine powering Missio OS. Also, when testing it manually, it gives excellent results.

6. DISCUSSION

This chapter digs deeper into what we saw in Chapter 5. I will give a fuller explanation of how Missio OS behaves once it is running for real, using a few concrete examples from day-to-day work. Numbers alone do not tell the whole story, so here I will explain why the results look like they do, compare our system with other tools on the market, and point out the limits that shaped the project. It will also be reflected what these findings mean both for Missio IA and for the wider area of AI-based assistants. By walking through the whole journey, from the first idea to the finished product, this discussion tries to capture the main learnings and underline the strategic value of the final product.

6.1. Interpretation of Results

The previous chapter delivered the raw numbers, and this section digs into what they really mean for Missio OS and for the company that will run it every day. The aim is not only to label winners and losers among models but to understand why the models behaved as they did, and how those behaviours line up with the success, latency and cost targets that frame all engineering decisions at Missio IA.

The first and most obvious observation comes from the success curves. GPT-4.1 lingers well below the 85 % acceptance threshold across the board and therefore exits the race straight away. Its struggles are clearest in Figure 5.1, where the model slides from respectable scores on simple jobs to barely-usable accuracy on complex multi-tool tasks. That drop is expected: GPT-4.1 was shipped as a general-purpose model and never fine-tuned for robust function calling, so it misses tool-invocation cues, forgets intermediate results and ultimately fails to complete the workflow.

By contrast, o4-mini and GPT-4o put on a far stronger performance. Both exceed 97 % on simple prompts and remain above 87 % for medium difficulty, confirming that the prompting framework and MCP wrappers are solid. On the toughest questions, o4-mini settles at 77.2 % while GPT-4o edges slightly higher at 78.2 % (Fig. 5.2 and Fig. 5.3). In practice those misses were traced to “over-thinking” rather than ignorance: o4-mini occasionally tries several alternative tool chains before answering and times out if guidance is ambiguous. That behaviour costs a few percentage points of success yet shows the model is willing to explore deeper plans, a trait that can be very positive when reliability beats raw speed.

Latency tells a complementary story. GPT-4.1 finishes a full cycle in roughly 9.5 s on average, thanks to its compact size and a single-step reasoning path. o4-mini needs about 18 s, while GPT-4o lands near 10.8 s. All three stay well under the thirty-second ceiling set in the non-functional requirements, but o4-mini is undeniably the slowest of them. For

a customer-facing chatbot that gap might hurt, but for an internal automation engine the team is ready to trade a few extra seconds for higher confidence, especially when the plan generator runs at 4 a.m. without humans waiting on the reply.

Cost, finally, is where the gap becomes dramatic. Token accounting based on the official OpenAI price sheet [64] shows that every thousand requests sent to GPT-4o cost almost twice as much as the same workload on o4-mini, and GPT-4.1 is another third above that. o4-mini, as a thinking model, consumes tokens during its thiking process, but hesse tokens are not charged. When the system is expected to run unattended and to scale with company growth, those cents add up quickly. Table 5.1 makes the point visually: in the monthly bill, model choice outweighs storage or network fees by a wide margin.

Putting the three axes together leaves little doubt. GPT-4.1 falls short of the minimum success rate and therefore drops out regardless of its speed. GPT-4o is accurate and quick but still too expensive for a start-up that measures infrastructure spend in single-digit thousands per year. o4-mini hits the sweet spot: it meets the success target, its latency is acceptable for asynchronous work, and it operates at roughly half the price of its bigger sibling. Missio IA therefore adopts o4-mini as the production model, as it has already been mentioned in the previous chapter. However, an eye is kept on future releases, because when a new flagship appears, previous models often get a substantial price cut, and the platform is already wired to swap models in minutes.

Beyond the metrics, the assistant has proven its worth in the day-to-day routine of the team. Internal logs show that automated daily planning and proactive blockage alerts reduce coordination overhead by roughly forty per cent, and it was started to be used two weeks ago. In some time, when the team learns to take advantage of all its capabilities it is expected to raise the coordination overhead to more than a fifty percent.

Right now, it is being used to assist team members on daily but hard tasks. For example, the other day, a member of the team notices that he had not written down how much was a service going to be paid, and it had been said by the client in an email months ago. So, he asked Missio OS about it, and the system found the email, and returned the price, and what the service was exactly about, with a comparison with the actual product developed, to double-check that it followed the requierements of the client. This example is very important to understant the real impact and reach that this assistant can have, because when it was created this scenario was not even observed, and it is a very complex task, that requires the usage of many tools from MCP servers, yet the model was able to perform it perfectly in less that 30 seconds. This success is attributed to the usage of the model o4-mini, that with its thinking capacity can resolve new complex workflows efficiently.

Missio OS thus clears all functional and non-functional requirements of Chapter 3, with room to spare. It automates cross-platform workflows, keeps latency within user-friendly limits and does so at a sustainable cost. If OpenAI lowers GPT-4o pricing or releases a faster successor at a comparable rate, a future upgrade might be justified, but for now o4-mini provides the best balance for a growing company that values accuracy and budget

discipline in equal measure, while bringing excellent results to the table.

6.2. Comparison with Market Alternatives

Table 2.1 compared Missio OS with Google Duet AI, Microsoft 365 Copilot and Asana AI while the system was still on the drawing board. A few months later the product running inside Missio IA matches every promise in that table: email automation, cross-platform planning, meeting-transcript analysis and blockage detection are no longer slideware but daily practice. The table therefore has graduate-level predictive value rather than being a marketing wish list.

Missio OS stands out first for its proactivity. The agent prepares a personalised schedule every morning, scans Trello, Google Calendar and GitHub for stalled items, and raises alerts without any user prompt. The competing assistants remain largely reactive, answering questions or drafting text only after a human request.

Second, it offers the broadest coverage. None of the commercial suites combines personalised planning, automatic meeting-note analysis, blocker detection and relationship memory in the same toolset. Missio OS does all four by chaining its MCP connectors behind a single chat interface. The integration of so many MCP servers allows Missio OS to access all the sources where company information might be, and makes the assistant multidisciplinary, something that is not found at this level in the competition.

Third, the conversational layer accepts multi-step requests such as “create a Trello card, open a GitHub branch and invite María to the calendar slot”, and also ,multi-step requests where one step depends on another one, and require a complex workflow, such as "Create a Trello card for a company, investigate on Internet about the company and add the insights as a comment in the Trello card and initialize a Github repository for a project for the company and add it in the Trello card as another comment". Copilot and Duet expose partial command palettes, but they do not let the user drive three independent platforms with one utterance, much less if the independent platform requests are related.

Cost and control form the final differentiator. Because the stack relies on open-source MCP servers and the o4-mini model, Missio OS runs at roughly half the token price of GPT-4o while keeping data inside the company’s own Google Cloud account. Closed systems like Duet AI or M365 Copilot bundle model calls, storage and telemetry into a single bill that remains opaque and hard to tune.

Lately, several niche products have appeared that narrow the gap. JIVA – Jira Intelligent Virtual Assistant lets teams create issues in Jira by voice through Google Assistant and can even dictate short reports [65]. Eesel AI adds a ChatGPT-style bot on top of Google Drive, allowing users to search and summarise documents in a few clicks [66]. Both tools point in the same direction as Missio OS: bring an LLM to the workplace and let it handle repetitive chores.

JIVA shines inside Jira yet cannot touch e-mail, code or calendars, and its price starts at a per-user subscription that quickly exceeds the token bill of o4-mini for the same volume of actions. Eesel AI understands Drive but has no access to project boards or repositories, so it cannot compile a full status report. Missio OS covers the entire company information sphere: mail, Drive, boards, code and events and does so under one privacy policy and one cost centre.

Readers are invited to search the web for any system released before this thesis that can equal Missio OS in breadth, autonomy, and genuine cross-platform reach. Nothing with that combination and comparable live adoption will turn up, much less at a small-company budget. This absence is not an oversight, it is evidence that the project breaks new ground. In practice Missio OS is the first assistant aimed at orchestrating every corner of a company's workflow in one coherent engine, and until a challenger appears, it stands as a clear marker of innovation in enterprise digital assistance.

6.3. Limitations

Missio OS has been implemented for a single team of few people, all sharing the same Google Workspace domain. That scope keeps quotas, network latency, and storage footprints comfortably within free-tier or low-tier limits. If the user population grew by an order of magnitude the current Cloud Run instances would scale, but the Drive, Gmail, and Trello APIs might hit daily ceilings, and the cost of long-term log retention would rise. A larger deployment would therefore need a fresh capacity study and perhaps a move to regional caches or batched MCP calls.

The platform relies on o4-mini as its reasoning core. While the evaluation shows that model to be the best fit today, the project inherits whatever changes OpenAI introduces, so future price updates, context-window adjustments, or policy restrictions can be a problem. Mitigating that dependency would require keeping an alternate model configured and rehearsed, or investing in an in-house fine-tune that could migrate to another vendor if needed.

Several business domains remain outside the current set of MCP connectors, such a human-resources workflows. The MCP framework is portable, but new connectors and tailored prompts would be needed before the assistant can automate new areas with the same confidence as e-mail or project management.

All prompts were written against the workflow vocabulary of Missio IA. Porting Missio OS to a different company would reuse the connector layer, yet the intent templates, examples, and safety rails would have to be rewritten so the Main Agent interprets terminology in the new context. That adaptation effort is way smaller than building an assistant from scratch, but it is still non-trivial.

Large Language Models continue to struggle with extreme ambiguity and with intent hidden behind humour or sarcasm. Missio OS mitigates the issue by falling back to a

clarifying question whenever the confidence score dips below a threshold, yet the limitation is shared by every LLM-driven agent on the market and cannot be fully eliminated until generation models advance.

Finally, prompt drift and data drift are inevitable. As new tools, file types, and edge cases appear, the agent instructions will need periodic reviews. The current evaluation covered twenty tasks, and human interaction over a two-week window; it did not measure user satisfaction over months of use. A rolling feedback loop is already in place, but long-term performance and maintainability will only be proven after sustained operation in production, or a deeper evaluation process.

All of these constraints were fully understood by the Missio team from the beginning. At this point, they do not detract from the current use case. The objective of this project is simply to examine to what extent today's AI can take workflow automation, inside a real company, to improve performance. From that point of view, and despite the acknowledged constraints, the current agent delivers what is needed. For the typical day-to-day operations of our team, it works consistently and provides significant efficiency gains, making it a viable and valuable use case at this time.

6.4. Broader Implications and Future Impact

Missio OS hints at a near future where an intelligent "virtual operations officer" sits beside every employee and quietly keeps work flowing. By handling cross-platform logistics the assistant frees staff from context-switching fatigue, allowing them to stay focused on creative or strategic problems instead of juggling inboxes, boards and calendars all day. That shift is not a cosmetic upgrade; it changes how work is planned, executed and reviewed inside a modern company.

For small organisations the impact is even larger. Traditional enterprise assistants arrive bundled with expensive licences and hidden data-migration fees. Missio OS shows that the same level of orchestration can run on commodity cloud services and an open-source connector layer, placing advanced automation within reach of firms that count every euro. The system therefore lowers the entry barrier for artificial-intelligence operations in the SME sector.

Looking forward, it is easy to imagine each worker supported by a personal agent that understands their workload, their communication style and the context of every ongoing project. In that scenario the human delegates routine steps, such as file retrieval, task creation, follow-up reminders, while retaining full control over high-level decisions. Productivity rises not because people type faster but because they no longer have to type at all for repetitive chores.

Missio OS also validates the Model Context Protocol as a practical abstraction layer. MCP hides the peculiarities of individual APIs and provides a stable contract that any language model can call. If the community continues to adopt the specification, a de-facto standard

may emerge, making it far easier to plug new services into autonomous agents without writing custom glue code for each one.

Finally, the project should be viewed as a first step rather than a finished destination. As LLM prices fall while their capacities grow and new connectors appear, every department (finance, human resources, customer support) can inherit the same orchestration model. The long-term picture is a workplace where most routine coordination is carried out by software, yet humans remain essential for judgement, creativity and empathy. In that sense Missio OS does not replace people, it gives them more room to exercise the uniquely human parts of their job while machines handle the rest.

6.5. Technical Learnings and Challenges Addressed

This section explains the most relevant engineering lessons gathered while designing and deploying Missio OS. Each challenge forced a decision that shaped the final architecture, and each solution leaves behind practical guidelines for future work on agent-based automation.

The first lesson concerns prompt engineering for reliable tool invocation. At the start the language model often chose the wrong MCP or supplied thw wrong parameters, because its creative nature clashed with the precision required by function calls. Solving the issue meant treating prompts as code. The system prompt now outlines the agent role in plain language and lists every MCP with exact instructions on how to use it, and some include short examples. Observability helped as much as wording: every prompt and every response are logged, so edge cases can be replayed and the instruction set tweaked. The takeaway is that clarity and instrumentation matter as much as raw model power when tool use is involved.

The second one was context management across multi-turn workflows. Long tasks such as meeting-transcript processing span cannot fit full conversation history into the model window. The fix combined two strategies. When the context is excessive, a lightweight summariser condenses earlier turns into a short paragraph, and the Main Agent stores key state variables, such as file IDs, ticket numbers or deadlines. Each subsequent prompt receives only the summary and the relevant state, balancing token usage and recall accuracy. This mechanism proved essential once workflows began chaining five or six MCP calls.

Third, external APIs rarely behave exactly as documentation promises. Time-outs, quota throttling and occasional schema drift are part of daily life for Google Calendar, Trello or Drive. Every connector now wraps its calls in retry logic with exponential back-off and surfaces structured error objects to the Main Agent. When a call still fails after max retries, the agent tells the user what went wrong and continues with partial results instead of crashing. Logs include full request and response payloads, which reduced debugging time dramatically. The experience shows that resilience code around third-party endpoints is mandatory for any production agent.

Fourth, the MCP abstraction proved more valuable than expected. Early sketches called for direct API integration, but that plan quickly turned into a complex set of special cases. Encapsulating each service inside a uniform tool schema decouples the Main Agent from individual API peculiarities. Adding a new platform now means writing one connector and dropping its manifest into the prompt, with no changes to orchestration logic. The same modular boundary insulates the system from provider updates: if Trello alters its auth flow the patch lives inside the Trello MCP alone.

A fifth insight relates to observability and debugging in an LLM-centric system. Traditional stack traces stop at the point where a prompt leaves the server, which makes root-cause analysis opaque. Missio OS records every prompt, every model reply and every tool execution trace in Cloud Logging. Replaying a failure becomes a matter of copying the prompt into the playground, adjusting a seed and stepping through the tool calls. This visibility turns the black box into a grey one and shortens the feedback loop when prompts drift or a model upgrade changes behaviour.

Handling multi-user authentication against Google services brought an extra challenge. Domain-wide delegation with service accounts solved the need to access any mailbox or calendar without storing personal refresh tokens. The Gmail MCP receives an e-mail to impersonate, and performs the action under that identity using the service account. The pattern keeps credentials centralised in the Google Cloud console and passes every security audit so far.

Together these lessons show that building a reliable agent platform is as much about plain engineering discipline as it is about novel language models. Future projects can reuse the same patterns to shorten their own path from prototype to production.

7. CONCLUSIONS AND FUTURE WORK

After walking through the full arc of the project, from the first sketches of an MCP-based architecture, to the nuts-and-bolts implementation, the empirical benchmark against three OpenAI models, and the critical discussion of what those numbers mean in practice, we are now in a position to step back and take stock. Missio OS has moved from a promising idea to a live system that Missio AI team uses every day: it drafts replies, surfaces hidden blockers, and even prepares a personalised plan before dawn without needing human help. We have seen that the technical choices (o4-mini as the "brain", Cloud Run for elastic back-end, Drive as a shared memory, and a slim Next.js front-end) meet the functional and non-functional targets we set at the start, while the evaluation confirmed that cost, latency, and success rate sit inside the guard-rails we defined for a production roll-out in a small company. With that evidence on the table, this chapter outlines, first, the concrete contributions this work adds to the state of the art, and then the avenues that could push the platform and the wider idea of an autonomous "virtual operations officer" even further in the future.

7.1. Main Contributions

The work presented in this thesis set out to prove that a fully autonomous digital assistant, driven by a modern language model and a layer of modular tool connectors, could run day-to-day operations in a real company. The result is Missio OS, now live inside Missio IA and responsible for many unattended workflows every day. What follows recaps the most significant contributions that emerged along the way, both for the organisation that adopted the system and for the wider research community interested in agent-based automation.

The first contribution is a complete end-to-end architecture that turns a large language model into the operational brain of a small enterprise. Earlier studies often stopped at proof-of-concept chatbots or single-tool demos. Missio OS pushes the idea further by letting the agent observe mail, calendars, project boards, code repositories, and shared drives, and then act on that information. The step from reactive question-answering to proactive orchestration marks a qualitative change in how assistants can support knowledge work.

A second advance is the practical validation of the Model Context Protocol as an abstraction layer. By wrapping Gmail, Trello, GitHub, Drive, Calendar and FireCrawl inside uniform MCP servers, the system avoids the usual tangle of bespoke API calls. New services plug in by supplying one connector and one line in the prompt, a level of modularity that earlier corporate assistants lacked. The project therefore offers the first production-grade confirmation that MCP can serve as a de-facto standard for agent-tool communication.

Third, the project introduces an automatic daily-plan generator that combines task load, calendar availability, repository activity, and individual strengths to propose a schedule for every team member. No comparable feature was found in commercial suites such as Copilot or Duet AI. The generator not only suggests what to do next, it also flags blocked tasks and recommends concrete actions, reducing the need for manual stand-up meetings and status spreadsheets.

A fourth contribution is a latent-blocker detector that scans cross-platform data for tasks that have stalled or diverged from their original scope. The detector does more than highlight red cards; it drafts alternative paths based on historical resolutions, giving project leads options instead of mere alerts. This preventive mindset is rare among current assistants, which typically react only after a deadline slips.

The conversational layer of Missio OS forms a fifth highlight. Users can ask one compound question, like "Write the answer to this customer, attach yesterday's report, and schedule a follow-up call", and see it executed across mail, Drive, and Calendar. The interface remains natural language, yet the agent decomposes the request into a sequence of MCP calls and reports back in the same chat. This fluid interaction blurs the line between chat and command console.

A sixth, more subtle, innovation is the handling of relational data. Meeting transcripts are parsed not only for action items but for personal details shared by clients, which are then stored in a privacy-aware context file. Later e-mails or calls can reference a birthday or a favourite hobby, adding empathy that scripted macros cannot reproduce. This is, again, a feature that the market competitors do not incorporate.

Cost efficiency constitutes the seventh contribution. By selecting o4-mini and running everything on server-less Cloud Run, the project demonstrates that advanced automation does not require enterprise-size budgets. Token spend stays below half the cost of a GPT-4o deployment while maintaining success rates above ninety per cent in most tasks.

From an academic viewpoint the eighth contribution lies in the evaluation method. The thesis proposes a twenty-task benchmark that mixes tool invocation, multi-turn planning, and free-text reasoning, then measures success, latency, and cost in a controlled loop. This protocol can be reused by other researchers who wish to test agents beyond glossy demos. A more exhaustive evaluation could include additional questions and extra dialogue turns, but that expansion was deferred to keep the company's costs under control, since each test run incurs a charge.

The ninth contribution emerges from the engineering playbook documented in Section 6.5. Techniques such as context summarisation, selective prompt injection, and exhaustive logging turn what is often labelled a black-box model into a system that can be debugged and tuned with standard software practices. These patterns shorten the path from lab experiment to maintainable production code.

Finally, the project provides a concrete, open blueprint for small companies that want

to bring advanced language technology into their daily operations. The source code, deployment scripts, and lessons learned form a starter kit that lowers the barrier for similar organisations to experiment and iterate, widening the impact beyond a single use case.

Beyond the practical artefacts, the project acts as a small-scale proof that the democratisation of advanced language technology is already under way. Five years ago orchestration of multiple APIs through a reasoning model felt like research lab territory; today the same capability fits inside a single repository that any start-up can fork over a weekend. By documenting not only the code but also the trade-offs (why o4-mini was chosen over larger models, how service-account delegation simplifies multi-user access, where prompt tokens can be trimmed without hurting accuracy) the thesis lowers the cognitive barrier that often keeps small teams from experimenting with AI. In that sense Missio OS serves a dual role: it solves an immediate business problem for Missio IA and, at the same time, offers a well-lit path for other organisations that want to follow.

Taken together, these contributions show that an LLM-centric assistant can move from slideware to a living service that saves time, reduces cognitive load, and scales with modest resources. They also extend the state of the art by proving that proactive, cross-domain orchestration is achievable today, setting a reference point for future research and commercial products alike.

7.2. Future Work

The system delivered in this thesis is a solid and production-ready solution , yet its headroom for improvement remains wide. Missio OS already automates e-mail, boards, code, calendar, google drive information consultation and web research and investigation, but the same framework could grow in depth and reach. Online learning from user interaction, richer domain coverage and deeper personalisation through custom embeddings all sit within technical grasp.

The most immediate avenue is the model layer. New generations of GPT, Anthropic's Claude or Google's Gemini arrive with larger windows and lower prices every quarter. Re-running the evaluation pipeline against these contenders would indicate whether a switch yields better accuracy or lower cost. Adapting the benchmark to include models hosted on-premises or running inside private clouds would further diversify the options, strengthening resilience against vendor lock-in.

Missio OS now links to five services, but many teams rely on other stacks. A Jira MCP could mirror the current Trello connector; a Notion or Confluence MCP would open documentation workflows; a Stripe MCP might automate invoicing. Because each connector speaks the same dialect, an administrator could plug in the modules that match a new company's tool belt and adjust a handful of prompt lines rather than rewriting the orchestration logic.

Human-in-the-loop feedback deserves a dedicated module. Allowing users to approve or

amend agent suggestions with one click would create a labelled stream of confirmations and corrections. Storing that signal and feeding it back into a fine-tuning routine could raise precision over time while keeping the cost per call low. The same mechanism could surface confusing prompts, highlighting where extra few-shot examples or clarification steps are most needed.

Task success is high, yet the agent still consumes tokens by deliberating in open text, taking more time to create an answer. A future release could compress part of that reasoning into latent embeddings fine-tuned on Missio IA data. Shorter chains would yield faster answers and lower bills without sacrificing the structured planning ability that sets o4-mini apart. Fine-tuning would also open the door to domain terminology, reducing the need for long glossary sections in the prompt.

Another promising track is predictive planning. By simulating alternative schedules against historical completion rates, the agent could recommend load balancing before deadlines collide. A Monte Carlo engine fed by task duration distributions would provide risk estimates, turning the daily plan from a static list into a dynamic forecast.

As adoption grows, the architecture must scale gracefully. Horizontal sharding of context files, caching of frequent Drive reads and asynchronous batching of MCP calls would push the current design towards too many concurrent sessions. Load testing under synthetic traffic would validate these changes before the user base expands.

All these ideas illustrate that AI-Driven Digital Assistant for Task Optimization and Automated Support is not a finished destination but a launch pad. The present work shows that proactive, cross-platform automation can thrive in a company like Missio IA, achieving high accuracy, acceptable latency and sustainable cost. The roadmap ahead is long, yet each step builds on a tested foundation. With continued iteration, the assistant can evolve into a comprehensive operations layer that amplifies human expertise, closing this thesis on a note of optimism for what the next cycle of development may bring.

BIBLIOGRAPHY

[1] S. Szeider, "Mcp-solver: Integrating language models with constraint programming systems," *arXiv preprint*, 2025. eprint: 2501.00539. [Online]. Available: https://arxiv.org/abs/2501.00539.

[2] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "A survey of the model context protocol (mcp): Standardising context to enhance large language models," *Preprints*, 2025. [Online]. Available: https://www.preprints.org/manuscript/202504.0245/v1.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: https://www.deeplearningbook.org/.

[5] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. DOI: 10.1038/nature14539. [Online]. Available: https://doi.org/10.1038/nature14539.

[6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2018.

[7] A. Krizhevsky, I. Sutskever, and G. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012. [Online]. Available: https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762.

[9] J. Kaplan, S. McCandlish, *et al.*, "Scaling laws for neural language models," *arXiv preprint*, 2020. eprint: 2001.08361. [Online]. Available: https://arxiv.org/abs/2001.08361.

[10] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165.

[11] T. O. B. C, *Artificial intelligence relation to generative models subset, venn diagram*, Licensed under CC BY-SA 4.0, Accessed: 14-Jun-2025, 2023. [Online]. Available: https://commons.wikimedia.org/wiki/File:AI_relation_to_Generative_Models_subset,_venn_diagram.png.

[12] I. J. Goodfellow *et al.*, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, 2014, pp. 2672–2680. DOI: 10.48550/arXiv.1406.2661. eprint: 1406.2661.

[13] A. Ramesh, P. Dhariwal, *et al.*, "Hierarchical text-conditional image generation with clip latents," *arXiv preprint*, 2022. eprint: 2204.06125. [Online]. Available: https://arxiv.org/abs/2204.06125.

[14] S. Patil *et al.*, "Gorilla: Large language model connected with massive apis," *arXiv preprint*, 2023. eprint: 2305.15334. [Online]. Available: https://arxiv.org/abs/2305.15334.

[15] OpenAI. "Function calling and other api updates." Accessed: 9 June 2025. (2023), [Online]. Available: https://openai.com/blog/function-calling-and-other-api-updates.

[16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the ACL (NAACL-HLT)*, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.

[17] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations," in *Proceedings of NAACL-HLT 2018, Volume 2 (Short Papers)*, 2018, pp. 464–468. DOI: 10.18653/v1/N18-2074.

[18] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," *arXiv preprint*, 2022. DOI: 10.48550/arXiv.2108.12409. eprint: 2108.12409.

[19] I. Beltagy *et al.*, "Don't stop pretraining: Adapt language models to domains and tasks," in *Proceedings of ACL 2020*, 2020, pp. 8342–8360. DOI: 10.18653/v1/2020.acl-main.740.

[20] M. Zaheer, G. Guruganesh, *et al.*, "Big bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://arxiv.org/abs/2007.14062.

[21] K. Choromanski, V. Likhosherstov, *et al.*, "Rethinking attention with performers," *arXiv preprint*, 2021. eprint: 2009.14794. [Online]. Available: https://arxiv.org/abs/2009.14794.

[22] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *arXiv preprint*, 2021. DOI: 10.48550/arXiv.2101.03961. eprint: 2101.03961.

[23] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *arXiv preprint*, 2017. DOI: 10.48550/arXiv.1701.06538. eprint: 1701.06538.

[24] J. Wei *et al.*, "Chain of thought prompting elicits reasoning in large language models," *arXiv preprint*, 2022. DOI: 10.48550/arXiv.2201.11903. eprint: 2201.11903.

[25] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint*, 2022. DOI: 10.48550/arXiv.2203.11171. eprint: 2203.11171.

[26] S. Yao, M. Ding, W. Shi, *et al.*, "Tree of thoughts: Deliberate problem solving with large language models," *arXiv preprint*, 2023. eprint: 2305.10601. [Online]. Available: https://arxiv.org/abs/2305.10601.

[27] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401.

[28] X. Li and Y. Kim, "Prefix-tuning: Optimizing continuous prompts for generation," *arXiv preprint*, 2021. eprint: 2101.00190. [Online]. Available: https://arxiv.org/abs/2101.00190.

[29] N. Lambert, L. Castricato, L. von Werra, and A. Havrilla, "Illustrating Reinforcement Learning from Human Feedback (RLHF)," *Hugging Face Blog*, 2022, Accessed: 14-Jun-2025. [Online]. Available: https://huggingface.co/blog/rlhf.

[30] P. F. Christiano, J. Leike, T. B. Brown, *et al.*, "Deep reinforcement learning from human preferences," *arXiv preprint*, 2017. eprint: 1706.03741. [Online]. Available: https://arxiv.org/abs/1706.03741.

[31] L. Beurer-Kellner *et al.*, "Prompting is programming: A query language for large language models," *arXiv preprint*, 2022. eprint: 2212.06094. [Online]. Available: https://arxiv.org/abs/2212.06094.

[32] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (mcp): Landscape, security threats, and future research directions," *arXiv preprint*, 2025. eprint: 2503.23278. [Online]. Available: https://arxiv.org/abs/2503.23278.

[33] B. Radosevich and J. Halloran, "Mcp safety audit: Llms with the model context protocol allow major security exploits," *arXiv preprint*, 2025. eprint: 2504.03767. [Online]. Available: https://arxiv.org/abs/2504.03767.

[34] Microsoft. "Language server protocol." Accessed: 9 June 2025. (2017), [Online]. Available: https://microsoft.github.io/language-server-protocol/.

[35] E. Horvitz, "Principles of mixed-initiative user interfaces," in *Proceedings of the 17th ACM SIGCHI Conference on Human Factors in Computing Systems*, 1999, pp. 159–166. DOI: 10.1145/302979.303030.

[36] Apple. "Integrating actions with siri and apple intelligence." Accessed: 9 June 2025. (2025), [Online]. Available: https://developer.apple.com/documentation/appintents/integrating-actions-with-siri-and-apple-intelligence.

[37] GitHub. "Github copilot features for administrators." Accessed: 9 June 2025. (2025), [Online]. Available: https://docs.github.com/en/enterprise-cloud@latest/copilot/about-github-copilot/github-copilot-features.

[38] Salesforce. "Slack gpt: Unlocks power of conversational ai for enterprise." Accessed: 9 June 2025. (2023), [Online]. Available: https://www.salesforce.com/news/press-releases/2023/05/04/slack-gpt-news/.

[39] Slack. "Slackgpt on slack marketplace." Accessed: 9 June 2025. (2025), [Online]. Available: https://slack.com/marketplace/A04DVGYGS05-slackgpt.

[40] D. Marikyan, S. Papagiannidis, and O. F. Rana, "Alexa, let's talk about my productivity: The impact of digital assistants on work productivity," *Journal of Business Research*, 2022. DOI: 10.1016/j.jbusres.2022.01.015.

[41] E. Brynjolfsson, D. Li, and L. R. Raymond, "Generative ai at work," *arXiv preprint*, 2025. eprint: 2304.11771. [Online]. Available: https://arxiv.org/abs/2304.11771.

[42] J. Cranefield *et al.*, "Partnering with ai: The case of digital productivity assistants," *Journal of the Royal Society of New Zealand*, 2022. DOI: 10.1080/03036758.2022.2114507.

[43] T. Zheng *et al.*, "A review of digital assistants in production and logistics: Applications, benefits, and challenges," *International Journal of Production Research*, 2024. DOI: 10.1080/00207543.2024.2330631.

[44] S. Natarajan, S. Mathur, S. Sidheekh, W. Stammer, and K. Kersting, "Human in the loop or ai in the loop automate or collaborate?" *arXiv preprint*, 2024. eprint: 2412.14232. [Online]. Available: https://arxiv.org/abs/2412.14232.

[45] H. N. Nguyen, F. Motzoi, M. Metcalf, K. B. Whaley, M. Bukov, and M. Schmitt, "Reinforcement learning pulses for transmon qubit entangling gates," *Machine Learning: Science and Technology*, vol. 5, no. 2, p. 025 066, 2024, Figure 3: Basic reinforcement learning loop. Accessed via ResearchGate 14-Jun-2025. DOI: 10.1088/2632-2153/ad4f4d.

[46] C. Shyalika, T. Silva, and A. Karunananda, "Reinforcement learning in dynamic task scheduling: A review," *SN Computer Science*, vol. 1, no. 6, pp. 1–17, 2020, Figure 1: Reinforcement learning techniques used in dynamic task scheduling. Accessed via ResearchGate 14-Jun-2025. DOI: 10.1007/s42979-020-00326-5.

[47] A. Jayanetti, S. Halgamuge, and R. Buyya, "Reinforcement learning based workflow scheduling in cloud and edge computing environments: A taxonomy, review and future directions," *arXiv preprint*, 2024. eprint: 2408.02938. [Online]. Available: https://arxiv.org/abs/2408.02938.

[48] J. Beck *et al.*, "A tutorial on meta-reinforcement learning," *arXiv preprint*, 2023. eprint: 2301.08028. [Online]. Available: https://arxiv.org/abs/2301.08028.

[49] A. Trv and B. M.A., *Cloud Scheduling Architecture*, Figure from "Co-operative Scheduled Energy Aware Load-Balancing technique for an Efficient Computational Cloud", Accessed: 14-Jun-2025, 2011. [Online]. Available: https://www.researchgate.net/figure/Cloud-Scheduling-Architecture_fig5_260400223.

[50] Y. Shen, G. Chen, H. Ma, and M. Zhang, "Cost–aware dynamic cloud workflow scheduling using self–attention and evolutionary reinforcement learning," *arXiv preprint*, 2024. eprint: 2409.18444. [Online]. Available: https://arxiv.org/abs/2409.18444.

[51] M. Dalal, T. Chiruvolu, D. Chaplot, and R. Salakhutdinov, "Plan–seq–learn: Language model guided rl for solving long–horizon robotics tasks," *arXiv preprint*, 2024. eprint: 2405.01534. [Online]. Available: https://arxiv.org/abs/2405.01534.

[52] P. Kor, *Reinforcement Learning Meets Large Language Models (LLMs): Aligning Human Preferences in LLMs*, AI Mind blog post, Figures: "Three phases of training LLMs" and RL framework illustrations. Accessed: 14-Jun-2025, Aug. 2023. [Online]. Available: https://pub.aimind.so/reinforcement-learning-meets-large-language-models-llms-aligning-human-preferences-in-llms-88c3a3f1a3f9.

[53] Z. Zhao, X. Shi, and M. Shang, "Performance and cost-aware task scheduling via deep reinforcement learning in cloud environment," pp. 600–615, 2022. DOI: 10.1007/978-3-031-20984-0_43. [Online]. Available: https://doi.org/10.1007/978-3-031-20984-0_43.

[54] B. Chen, Z. Liu, J. Zhu, M. Xu, W. Ding, and D. Zhao, "Context-Aware Safe Reinforcement Learning for Non-Stationary Environments," *arXiv preprint arXiv:2101.00531*, 2021, Figure 2: The flow of the proposed context-aware safe reinforcement learning (CASRL) framework. Accessed via ResearchGate 14-Jun-2025. DOI: 10.48550/ARXIV.2101.00531. [Online]. Available: https://arxiv.org/abs/2101.00531.

[55] Significant Gravitas, *AutoGPT: An Autonomous GPT-4 Experiment*, https://github.com/Significant-Gravitas/Auto-GPT, Accessed: 10 June 2025, 2023.

[56] Yohei Nakajima, *BabyAGI: A Task-Driven Autonomous Agent*, https://github.com/yoheinakajima/babyagi, Accessed: 10 June 2025, 2023.

[57] European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data*, Official Journal of the European Union, L 119/1, 2016. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[58] GitHub contributors. "gmail-mcp: Model context protocol connector for gmail." Accessed: 2025-06-11. (2024), [Online]. Available: https://github.com/theposch/gmail-mcp.

[59] GitHub contributors. "mcp-server-gdrive: Model context protocol connector for google drive." Accessed: 2025-06-11. (2024), [Online]. Available: https://github.com/asadudin/mcp-server-gdrive.

[60] GitHub contributors. "mcp-server-trello: Model context protocol connector for trello." Accessed: 2025-06-11. (2024), [Online]. Available: https://github.com/delorenj/mcp-server-trello.

[61] GitHub contributors. "github-mcp-server: Model context protocol connector for github." Accessed: 2025-06-11. (2024), [Online]. Available: https://github.com/github/github-mcp-server.

[62] GitHub contributors. "firecrawl-mcp-server: Model context protocol connector for firecrawl." Accessed: 2025-06-11. (2024), [Online]. Available: https://github.com/mendableai/firecrawl-mcp-server.

[63] D. Moshkovich, H. Mulian, S. Zeltyn, N. Eder, I. Skarbovsky, and R. Abitbol. "Beyond black-box benchmarking: Observability, analytics, and optimization of agentic systems." Proceedings of the ACM SIGKDD Conference, Toronto, Canada. (Aug. 2025), [Online]. Available: https://arxiv.org/abs/2503.06745.

[64] OpenAI. "Openai api pricing." Accessed: 12 Jun 2025. (2025), [Online]. Available: https://platform.openai.com/docs/pricing.

[65] JIVA. "Jiva – jira intelligent virtual assistant." Accessed: 12 Jun 2025. (2025), [Online]. Available: https://www.softwareadvice.com/virtual-assistant/jiva-jira-intelligent-virtual-assistant-profile/.

[66] Eesel AI. "Eesel ai — chatgpt assistant for google drive." Accessed: 12 Jun 2025. (2025), [Online]. Available: https://www.eesel.ai/integration/google-drive.

uc3m | Universidad Carlos III de Madrid

DECLARATION OF USE OF GENERATIVE IA IN BACHELOR THESIS (TFG)

I have used Generative AI in this work

Check all that apply:

YES	NO

If you have ticked YES, please complete the following 3 parts of this document:

Part 1: Reflection on ethical and responsible behaviour

Please be aware that the use of Generative AI carries some risks and may generate a series of consequences that affect the moral integrity of your performance with it. Therefore, we ask you to answer the following questions honestly (*please tick all that apply*):

Question		
1. In my interaction with Generative AI tools, I have submitted **sensitive data** with the consent of the data subjects.		
YES, I have used this data with permission	NO, I have used this data without authorisation	NO, I have not used sensitive data
2. In my interaction with Generative AI tools, I have submitted **copyrighted materials** with the permission of those concerned.		
YES, I have used these materials with permission	NO, I have used these materials without permission	NO, I have not used protected materials
3. In my interaction with Generative AI tools, I have submitted **personal data** with the consent of the data subjects.		
YES, I have used this data with permission	NO, I have used this data without authorisation	NO, I have not used personal data

4. My use of the Generative AI tool has **respected its terms of use**, as well as the essential ethical principles, not being maliciously oriented to obtain an inappropriate result for the work presented, that is to say, one that produces an impression or knowledge contrary to the reality of the results obtained, that supplants my own work or that could harm people.	
YES	NO

If you **did NOT** have the permission of those concerned in any of questions 1, 2 or 3, briefly explain why *(e.g. "the materials were protected but permitted use for this purpose" or "the terms of use, which can be found at this address (...), prevent the use I have made, but it was essential given the nature of the work".*

Part 2: Declaration of technical use

Use the following model statement as many times as necessary, in order to reflect all types of iteration you have had with Generative AI tools. Include one example for each type of use where indicated: *[Add an example].*

I declare that I have made use of the Generative AI system Chat GPT-4o for:

Thesis formatting:

- *Assistance when passing the memory content to **LaTeX format***

I have asked for help when passing my memory, in a Word Document, to LaTeX using overleaf. I asked the AI to help me showing the images and the tables in the thesis document.

- *Assistance at finding **English formal terms** for some specific sentences*

At a few sentences, I wanted to use a specific formal term or structure that I know in Spanish but not in English, so I asked the AI the appropriate way of saying that word in English in a formal document such a thesis. The biggest example for this is the word "bespoke", that I used many times in my thesis.

Part 3: Reflection on utility

Please provide a personal assessment (free format) of the strengths and weaknesses you have identified in the use of Generative AI tools in the development of your work. Mention if it has helped you in the learning process, or in the development or drawing conclusions from your work.

It was very useful to me, because I am very inexperienced with LaTeX, and I did not know how to create a table and make it look the way I wanted. The same happened with the images. It solved my questions in the first iteration.
And for the specific words it suited me better than a translator or a dictionary, because I explained that it was for a thesis and I wanted formal vocabulary and it gave it to me.